

# **Solana SVM Study Course**

## **Course Overview**

# Course Title

**Mastering Solana and SVM Development: From EVM to Solana**

## Duration & Level

- **Duration:** 16 weeks (part-time) / 8 weeks (full-time)
- **Level:** Intermediate to Advanced
- **Prerequisites:** JavaScript/TypeScript, basic blockchain concepts, familiarity with EVM development

## Learning Objectives

- Master Solana blockchain architecture and SVM (Solana Virtual Machine)
- Build Solana applications using NestJS
- Understand key differences between EVM and SVM paradigms
- Implement blockchain solutions
- Deploy and monitor Solana applications in production

# Course Repository Structure

```
solana-svm-study/
├── docs/
│   ├── COURSE.md
│   ├── STUDY.md
│   ├── TASKS.md
│   ├── MASTER-ITERATION.md
│   └── diagrams/
├── src/
│   ├── modules/
│   │   ├── accounts/
│   │   ├── transactions/
│   │   ├── tokens/
│   │   └── ...
│   ├── common/
│   ├── database/
│   └── shared/
├── infra/
├── scripts/
└── ...
```

# Course documentation  
# This course curriculum  
# Detailed study topics  
# Implementation tasks  
# Project philosophy  
# Architecture diagrams  
# Application source code  
# Feature modules  
# Account management  
# Transaction services  
# SPL token operations  
# Additional modules  
# Shared utilities  
# Database layer  
# Shared types  
# Infrastructure as code  
# Utility scripts  
# Configuration files

# Technology Stack

## Backend Framework

- **NestJS:** Progressive Node.js framework
- **TypeScript:** Type-safe JavaScript
- **TypeORM:** TypeScript ORM for databases

## Blockchain Integration

- **Solana Web3.js:** Solana blockchain interaction
- **@solana/web3.js:** Official Solana JavaScript SDK

## Database & Messaging

- **PostgreSQL:** Primary database
- **Apache Kafka:** Event streaming

# Development Environment

## Local Development

- Docker Compose for services
- Hot reload with NestJS CLI
- Integrated testing with Jest
- Code coverage reporting

## Production Deployment

- Kubernetes manifests
- CI/CD with GitHub Actions
- Multi-stage Docker builds
- Health checks and monitoring

# Course Modules Overview

## Core Modules

1. **Accounts & Programs** - Account management and PDAs
2. **Transactions & Instructions** - Transaction building and submission
3. **Token Standards** - SPL token operations
4. **Account Abstraction** - Smart accounts and PDAs
5. **Fee Mechanism** - Transaction fees and prioritization

## Advanced Modules

6. **Consensus & Validation** - Block validation and consensus
7. **Signing & Cryptography** - Digital signatures and keys
8. **MPC** - Multi-party computation protocols
9. **SVM** - Solana Virtual Machine integration



# Learning Approach

## Hands-on Implementation

- Build complete NestJS API for Solana integration
- Implement all major SVM features
- Real-world blockchain application development

## Progressive Complexity

- Start with basic concepts (accounts, transactions)
- Build up to advanced features (MPC, CPIs)
- End with production deployment and monitoring

## EVM to SVM Migration

- Direct comparisons between EVM and SVM concepts

# Assessment & Evaluation

## Implementation Tasks

- Complete all module implementations
- Pass comprehensive test suites
- Code review and optimization

## Project Milestones

- Database schema and migrations
- API endpoint implementation
- Integration testing
- Performance optimization

## Final Deliverables

# Success Metrics

## Technical Proficiency

- 80%+ test coverage
- Performance benchmarks met
- Security best practices implemented
- Production deployment successful

## Code Quality

- TypeScript strict mode compliance
- Clean architecture patterns
- Comprehensive error handling
- API documentation complete

# Thank You!

## Questions & Next Steps

- Review the [STUDY.md](#) for detailed topics
- Check [TASKS.md](#) for implementation guide
- Start with [Module 1: Accounts & Programs](#)

Happy Learning! 🚀

# **STUDY.md**

## **Solana and SVM Study Topics**

# Overview

This document outlines the study topics for mastering Solana and SVM (Solana Virtual Machine), with direct comparisons to EVM (Ethereum Virtual Machine) concepts where applicable.

Each topic includes key learning objectives and implementation considerations for this project.

# Core Concepts

## 1. Accounts and Programs

 [View Diagram](#)

- **Solana Equivalent:** Accounts (data storage) and Programs (smart contracts)
- **EVM Comparison:** Similar to contracts and storage, but accounts hold both code and data

## Key Topics - Accounts & Programs

- Account types: System accounts, program accounts, data accounts
- Rent exemption and account lifecycle
- Program Derived Addresses (PDAs) for deterministic addressing

**Implementation:** Account management API endpoints



## 2. Transactions and Instructions

 [View Diagram](#)

- **Solana Equivalent:** Transactions containing Instructions
- **EVM Comparison:** Transactions calling contract functions

## Key Topics - Transactions & Instructions

- Transaction structure and serialization
- Instruction format and execution
- Atomicity and transaction ordering
- Compute budget and prioritization

**Implementation:** Transaction building and submission services

### 3. Token Standards (SPL Tokens)

 [View Diagram](#)

- **Solana Equivalent:** SPL (Solana Program Library) Tokens
- **EVM Comparison:** ERC-20, ERC-721, ERC-1155

## Key Topics - Token Standards

- Token Program (SPL-Token)
- Associated Token Accounts (ATAs)
- Token metadata and extensions
- NFT standards (SPL-Token-2022)

**Implementation:** Token creation, transfer, and management APIs

## 4. Account Abstraction

 [View Diagram](#)

- **Solana Equivalent:** Program-controlled accounts and PDAs
- **EVM Comparison:** EIP-4337 Account Abstraction

## **Key Topics - Account Abstraction**

- Smart accounts via programs
- Programmable transaction authorization
- Session keys and delegated execution

**Implementation:** Abstracted account management system

## 5. Fee Mechanism

 [View Diagram](#)

- **Solana Equivalent:** Base fees + Priority fees
- **EVM Comparison:** EIP-1559 (base fee + priority fee)

## Key Topics - Fee Mechanism

- Fee calculation and prioritization
- Compute unit limits
- Fee markets and congestion handling

**Implementation:** Fee estimation and transaction optimization



## 6. Consensus and Validation

 [View Diagram](#)

- **Solana Equivalent:** Proof of Stake with Tower BFT
- **EVM Comparison:** Proof of Work (legacy) / Proof of Stake (post-Merge)

## **Key Topics - Consensus & Validation**

- Validator selection and rotation
- Leader schedule and block production
- Fork choice and finality

**Implementation:** Network monitoring and validation status APIs

## 7. Signing and Cryptography

 [View Diagram](#)

- **Solana Equivalent:** Ed25519 keypairs and signatures
- **EVM Comparison:** ECDSA secp256k1

## **Key Topics - Signing & Cryptography**

- Key generation and management
- Transaction signing workflows
- Multi-signature schemes
- Hardware wallet integration

**Implementation:** Secure signing services

## 8. Multi-Party Computation (MPC)

 [View Diagram](#)

- **Solana Equivalent:** Threshold signatures and distributed key generation
- **EVM Comparison:** Multi-sig wallets and threshold schemes

## Key Topics - MPC

- MPC protocols for secure signing
- Distributed key management
- Threshold cryptography implementations

**Implementation:** MPC wallet and transaction services

## 9. Solana Virtual Machine (SVM)

 [View Diagram](#)

- **Solana Equivalent:** Sealevel runtime and SVM
- **EVM Comparison:** EVM execution environment

## Key Topics - SVM

- Parallel transaction execution
- Runtime architecture
- Program compilation and deployment
- Gas metering and resource limits



## SVM Infrastructure

- Local test validator setup
- Network configuration (local/devnet/testnet/mainnet)
- RPC endpoint management
- Faucet integration for development
- Ledger state management
- CLI tools integration

**Implementation:** SVM integration and program execution APIs

## 10. Cross-Program Invocations (CPIs)

 [View Diagram](#)

- **Solana Equivalent:** Programs calling other programs
- **EVM Comparison:** Contract calls and DELEGATECALL

## Key Topics - CPIs

- CPI mechanics and security
- Program composition patterns
- Permission and access control

**Implementation:** Cross-program interaction services

## 11. Events and Logging

 [View Diagram](#)

- **Solana Equivalent:** Program logs and events
- **EVM Comparison:** Contract events and logs

## Key Topics - Events & Logging

- Event emission and parsing
- Log subscription and filtering
- Historical data indexing

**Implementation:** Event streaming and indexing system

## 12. Security and Best Practices

 [View Diagram](#)

### Key Topics - Security

- Common vulnerabilities and mitigations
- Program upgrade patterns
- Access control and authorization
- Audit considerations

**Implementation:** Security-focused API design

## 13. Development Tools and Frameworks

 [View Diagram](#)

### Key Topics - Development Tools

- Anchor framework for Rust programs
- Web3.js and JavaScript tooling
- Testing frameworks and methodologies
- Deployment and monitoring

**Implementation:** Development tooling integration

## 14. Network Architecture

 [View Diagram](#)

### Key Topics - Network Architecture

- Cluster types (mainnet, devnet, testnet)
- RPC nodes and load balancing
- Historical data access
- Network performance optimization

**Implementation:** Multi-network API support



## 15. Advanced Features

 [View Diagram](#)

### Key Topics - Advanced Features

- State compression and accounts database
- Versioned transactions
- Address lookup tables
- Program address introspection

**Implementation:** Advanced feature demonstrations

# Learning Objectives by Module

## Beginner Level

- Basic account operations
- Simple token transfers
- Transaction submission
- Network connection and RPC calls

## Intermediate Level

- Program development and deployment
- Complex token operations
- Multi-signature transactions
- Event handling and monitoring

## Advanced Level

- MPC implementations
- Cross-program invocations
- Performance optimization
- Security hardening

# Implementation Roadmap

1. Core account and transaction management
2. Token standards implementation
3. Signing and MPC services
4. Program interaction APIs
5. Event streaming and monitoring
6. Advanced features and optimizations

# Design Patterns

## Gang of Four (GoF) Patterns

### Factory Pattern

- **Application:** Transaction creation and account management
- **Implementation:** Abstract factory for different transaction types
- **Example:** `TransactionFactory.createTransfer()` vs `TransactionFactory.createTokenTransfer()`

## Strategy Pattern

- **Application:** Signing mechanisms and fee calculation
- **Implementation:** Pluggable signing strategies (Ed25519, MPC, hardware wallets)
- **Example:** `SigningStrategy` interface with multiple implementations

## Observer Pattern

- **Application:** Event monitoring and logging
- **Implementation:** Event listeners for transaction confirmations
- **Example:** `TransactionObserver` subscribing to WebSocket events



## Command Pattern

- **Application:** Transaction instructions and program invocations
- **Implementation:** Encapsulated instruction execution
- **Example:** `InstructionCommand` objects for various operations

## Adapter Pattern

- **Application:** Multi-chain integrations
- **Implementation:** Unified interface for EVM and Solana operations
- **Example:** `BlockchainAdapter` interface implementations

## Blockchain-Specific Patterns

### Token Factory Pattern

- **Application:** SPL token creation and management
- **Implementation:** Standardized token deployment with metadata
- **Example:** `TokenFactory.create()` with automatic ATA creation

## Multisig Wallet Pattern

- **Application:** Multi-signature transactions and MPC
- **Implementation:** Threshold signature schemes
- **Example:** `MultisigWallet` with configurable thresholds

## Oracle Pattern

- **Application:** External data feeds and price feeds
- **Implementation:** Decentralized data sources
- **Example:** `PriceOracle` for DeFi operations

## Bridge Pattern

- **Application:** Cross-chain asset transfers
- **Implementation:** Lock-and-mint mechanisms
- **Example:** BridgeService for EVM-Solana interoperability

## Solana-Specific Patterns

### Program Derived Addresses (PDAs)

- **Application:** Deterministic account creation
- **Implementation:** Cryptographic address derivation
- **Example:** `findProgramAddress([userPubkey, "escrow"], programId)`

## Associated Token Accounts (ATAs)

- **Application:** Automatic token account management
- **Implementation:** Deterministic token account addresses
- **Example:** `getAssociatedTokenAddress(mint, owner)`



## Cross-Program Invocation (CPI)

- **Application:** Program composition and modularity
- **Implementation:** Programs calling other programs
- **Example:** DEX program invoking token program for swaps

## Account Abstraction via Programs

- **Application:** Smart accounts and programmable authorization
- **Implementation:** Program-controlled accounts
- **Example:** SmartAccount program with session keys

## Event-Driven Architecture

- **Application:** Real-time blockchain monitoring
- **Implementation:** Program logs and transaction events
- **Example:** `ProgramEventListener` parsing transaction logs

## Resources

- [Official Solana Documentation](#)
- [Solana Web3.js Guide](#)
- [SPL Token Documentation](#)
- [Anchor Framework](#)
- [Solana Cookbook](#)

# Thank You!

## Next Steps

- Review [TASKS.md](#) for implementation details
- Explore the [architecture diagrams](#)
- Start building with the core modules

Happy Learning! 🚀

# **TASKS.md**

## **Implementation Tasks for Solana SVM Study Repository**

## Overview

This document outlines all implementation tasks required to complete the NestJS API for Solana and SVM integrations.

Tasks are organized by module and priority level, with detailed planning including story points, complexity assessment, skill levels, risks, opportunities, and security considerations.

## Task Format

Each task includes:

- **ID:** Reference to STUDY.md topic (e.g., STUDY-1 for Accounts and Programs)
- **Story Points:** Fibonacci scale (1, 2, 3, 5, 8, 13, 21)
- **Complexity:** Low/Medium/High
- **Level:** Beginner/Intermediate/Advanced
- **Risks:** Potential challenges and mitigations
- **Opportunities:** Benefits and learning outcomes
- **Security:** Critical security considerations



# Core Infrastructure Tasks

## Database and Persistence

| Task                               | ID      | SP | Complexity | Level        | Status                                                                                |
|------------------------------------|---------|----|------------|--------------|---------------------------------------------------------------------------------------|
| PostgreSQL connection with TypeORM | INFRA-1 | 3  | Low        | Beginner     |    |
| Account entity with relationships  | STUDY-1 | 2  | Low        | Beginner     |    |
| Token entity with metadata         | STUDY-3 | 2  | Low        | Beginner     |  |
| Transaction entity with status     | STUDY-2 | 3  | Low        | Beginner     |  |
| Database migrations                | INFRA-2 | 5  | Medium     | Intermediate |  |

## Message Queue Integration

| Task                         | ID       | SP | Complexity | Level        | Status |
|------------------------------|----------|----|------------|--------------|--------|
| Kafka client in NestJS       | INFRA-5  | 3  | Low        | Beginner     | ✓      |
| Transaction event publishing | STUDY-11 | 5  | Medium     | Intermediate | ✓      |
| Blockchain events consumer   | STUDY-11 | 8  | High       | Advanced     | ✓      |
| Dead letter queue            | INFRA-6  | 3  | Low        | Intermediate | ✓      |
| Message retry mechanisms     | INFRA-7  | 5  | Medium     | Intermediate | ✓      |

## Containerization

| Task                 | ID       | SP | Complexity | Level        | Status |
|----------------------|----------|----|------------|--------------|--------|
| Docker Compose setup | INFRA-8  | 5  | Medium     | Intermediate | ✓      |
| Redis caching layer  | INFRA-9  | 8  | High       | Advanced     | ✓      |
| Health checks        | INFRA-10 | 3  | Low        | Intermediate | ✓      |
| Prometheus + Grafana | INFRA-11 | 13 | High       | Advanced     | ✓      |

# Kubernetes Orchestration

| Task                  | ID    | SP | Complexity | Level        | Status |
|-----------------------|-------|----|------------|--------------|--------|
| Namespace creation    | K8S-1 | 1  | Low        | Beginner     | ✓      |
| PostgreSQL deployment | K8S-2 | 5  | Medium     | Intermediate | ✓      |
| Kafka cluster         | K8S-3 | 8  | High       | Advanced     | ✓      |
| Redis deployment      | K8S-4 | 3  | Low        | Intermediate | ✓      |
| NestJS application    | K8S-5 | 5  | Medium     | Intermediate | ✓      |
| Services and ingress  | K8S-6 | 5  | Medium     | Intermediate | ⌚      |
| Secrets management    | K8S-7 | 3  | Low        | Intermediate | ✓      |
| Resource limits       | K8S-8 | 3  | Low        | Intermediate | ✓      |
| ConfigMaps            | K8S-9 | 2  | Low        | Beginner     | ✓      |

# Accounts Module Implementation

## Basic Account Operations

| Task                      | ID      | SP | Complexity | Level        | Status |
|---------------------------|---------|----|------------|--------------|--------|
| Account creation endpoint | STUDY-1 | 2  | Low        | Beginner     | ✓      |
| Account retrieval         | STUDY-1 | 2  | Low        | Beginner     | ✓      |
| Balance queries           | STUDY-1 | 3  | Low        | Beginner     | ✓      |
| Account info fetching     | STUDY-1 | 3  | Low        | Beginner     | ✓      |
| Account updates           | STUDY-1 | 2  | Low        | Beginner     | ⌚      |
| Account deletion          | STUDY-1 | 3  | Medium     | Intermediate | ⌚      |

## Program Derived Addresses (PDAs)

| Task                   | ID      | SP | Complexity | Level        | Status                                                                               |
|------------------------|---------|----|------------|--------------|--------------------------------------------------------------------------------------|
| PDA generation service | STUDY-1 | 5  | Medium     | Intermediate |   |
| Address derivation     | STUDY-1 | 3  | Low        | Intermediate |   |
| PDA validation         | STUDY-1 | 2  | Low        | Intermediate |   |
| PDA account management | STUDY-4 | 8  | High       | Advanced     |  |

# Account Abstraction

| Task                      | ID      | SP | Complexity | Level    | Status |
|---------------------------|---------|----|------------|----------|--------|
| Smart account creation    | STUDY-4 | 13 | High       | Advanced | ✔      |
| Session key management    | STUDY-4 | 8  | High       | Advanced | ⌚      |
| Transaction authorization | STUDY-4 | 13 | High       | Advanced | ✔      |
| Batched transactions      | STUDY-4 | 8  | High       | Advanced | ⌚      |

# Tokens Module Implementation

## SPL Token Standards

| Task                    | ID      | SP | Complexity | Level        | Status |
|-------------------------|---------|----|------------|--------------|--------|
| Token creation endpoint | STUDY-3 | 3  | Low        | Beginner     | ✓      |
| Token metadata          | STUDY-3 | 5  | Medium     | Intermediate | 🕒      |
| Token minting           | STUDY-3 | 5  | Medium     | Intermediate | 🕒      |
| Token burning           | STUDY-3 | 3  | Low        | Intermediate | 🕒      |
| Supply management       | STUDY-3 | 3  | Low        | Intermediate | 🕒      |



## Associated Token Accounts (ATAs)

| Task                    | ID      | SP | Complexity | Level        | Status |
|-------------------------|---------|----|------------|--------------|--------|
| ATA creation/management | STUDY-3 | 3  | Low        | Beginner     | ✓      |
| ATA discovery           | STUDY-3 | 2  | Low        | Intermediate | ⌚      |
| ATA balance queries     | STUDY-3 | 2  | Low        | Beginner     | ✓      |
| ATA delegation          | STUDY-3 | 5  | Medium     | Intermediate | ⌚      |

## Token Operations

| Task                   | ID      | SP | Complexity | Level        | Status |
|------------------------|---------|----|------------|--------------|--------|
| Token transfers        | STUDY-3 | 3  | Low        | Beginner     | ✔      |
| Token approvals        | STUDY-3 | 5  | Medium     | Intermediate | ⌚      |
| Freeze/thaw operations | STUDY-3 | 3  | Low        | Intermediate | ⌚      |
| Account closure        | STUDY-3 | 2  | Low        | Intermediate | ⌚      |

# Transactions Module Implementation

## Basic Operations

| Task                         | ID      | SP | Complexity | Level        | Status |
|------------------------------|---------|----|------------|--------------|--------|
| Transaction creation/storage | STUDY-2 | 3  | Low        | Beginner     | ✓      |
| Transaction retrieval        | STUDY-2 | 2  | Low        | Beginner     | ✓      |
| Status tracking              | STUDY-2 | 5  | Medium     | Intermediate | ⌚      |
| History queries              | STUDY-2 | 5  | Medium     | Intermediate | ⌚      |

## Transaction Building

| Task                      | ID       | SP | Complexity | Level        | Status |
|---------------------------|----------|----|------------|--------------|--------|
| SOL transfer builder      | STUDY-2  | 3  | Low        | Beginner     | ✓      |
| Token transfer tx         | STUDY-2  | 5  | Medium     | Intermediate | ⌚      |
| Program invocation        | STUDY-10 | 8  | High       | Advanced     | ⌚      |
| Multi-instruction support | STUDY-2  | 8  | High       | Advanced     | ⌚      |

## Fee Management

| Task                     | ID      | SP | Complexity | Level        | Status                                                                               |
|--------------------------|---------|----|------------|--------------|--------------------------------------------------------------------------------------|
| Fee estimation service   | STUDY-5 | 5  | Medium     | Intermediate |   |
| Priority fee calculation | STUDY-5 | 3  | Low        | Intermediate |   |
| Fee optimization         | STUDY-5 | 8  | High       | Advanced     |   |
| Dynamic fee adjustment   | STUDY-5 | 5  | Medium     | Advanced     |  |

# Advanced Modules Status

## Multi-Party Computation (MPC)

-  Threshold signature generation
-  Distributed key generation
-  Key share management
-  Signature reconstruction
-  MPC wallet creation
-  Secure key distribution
-  MPC transaction signing
-  Recovery mechanisms

## Solana Virtual Machine (SVM)

-  Program entity and CRUD
-  Program deployment
-  Single program execution
-  Parallel transaction execution
-  Gas metering and tracking
-  SVM runtime monitoring
-  Local test validator
-  Multi-network support

## Cross-Program Invocations (CPIs)

-  CPI instruction builder
-  Program invocation utilities
-  CPI permission management
-  CPI error handling
-  Cross-program data sharing



## Event Monitoring

-  WebSocket connections
-  Event filtering/subscription
-  Event persistence
-  Event replay capabilities
-  Transaction confirmations
-  Account change notifications
-  Token transfer events
-  Program log monitoring

# Security Implementation

## Authentication & Authorization

-  API key authentication
-  JWT token management
-  Role-based access control
-  Rate limiting

## Cryptographic Security

-  Secure key management
-  Encrypted data storage
-  Secure random generation
-  Signature verification

# Testing & Quality Assurance

## Current Status

-  Basic test structure
-  Comprehensive unit tests (>80% coverage)
-  Mock services for blockchain
-  Test utilities and fixtures
-  API integration tests
-  Database integration tests
-  Kafka integration tests
-  End-to-end testing

# Priority Classification

## High Priority (P0) - Core Functionality

- Complete basic CRUD operations for accounts, tokens, transactions
- Implement transaction signing and submission
- Add comprehensive error handling
- Complete unit and integration testing

## Medium Priority (P1) - Enhanced Features

- Implement remaining MPC functionality
- Add event streaming capabilities
- Create advanced token operations
- Implement CPI framework

# Success Criteria

## Functional Requirements

- [x] All API endpoints return correct responses
- [x] Transaction operations execute successfully on Solana
- [x] Event streaming works in real-time
- [x] MPC operations complete securely

## Non-Functional Requirements

- [x] API response time < 500ms for cached requests
- [x] API response time < 2s for blockchain operations
- [x] 99.9% uptime for core services
- [x] >80% test coverage maintained

# Implementation Progress

## Completed Tasks: ~85%

- Core infrastructure fully deployed
- Basic CRUD operations implemented
- Advanced features (MPC, SVM, CPI) completed
- Security and authentication in place
- Event streaming and monitoring active

## Remaining Tasks: ~15%

- Enhanced token operations
- Advanced transaction features
- Comprehensive testing suite
- Performance optimizations

# Project Status: NEARLY COMPLETE

## Major Milestones Achieved

-  Full NestJS API with Solana integration
-  Kubernetes production deployment
-  Advanced cryptographic features (MPC)
-  Complete SVM implementation
-  Event-driven architecture
-  Security best practices
-  Monitoring and observability

**Ready for Production Deployment!** 

# Thank You!

## Next Steps

- Complete remaining P0 tasks
- Implement comprehensive testing
- Performance optimization
- Production deployment preparation

**The Solana SVM Study project is nearly complete! 🎉**



# Module 1: Accounts and Programs

## Core Concepts in Solana

# Accounts vs Programs

## Solana's Account Model

- **Accounts:** Data storage containers (like database records)
- **Programs:** Executable code (like smart contracts)
- **Key Difference:** Accounts can hold both data AND code

## EVM Comparison

- **EVM:** Contracts hold code, separate storage
- **SVM:** Accounts are unified - they can be programs OR data

# Account Types

## 1. User Accounts (External Wallets)

- Controlled by private keys
- Hold SOL and tokens
- Can sign transactions
- Created by System Program

## 2. Program Accounts

- Contain executable bytecode
- Immutable once deployed
- Executed by Solana runtime
- Cannot sign transactions directly

## Account Types (continued)

### 3. Program Derived Addresses (PDAs)

- Deterministically derived from seeds + program ID
- No private key exists
- Controlled by programs
- Enable program-controlled accounts

### 4. System Accounts

- Built-in Solana programs
- System Program, SPL Token Program, etc.
- Handle core blockchain operations

# Account States

## Account Lifecycle

- **Uninitialized:** No data, zero balance
- **Active:** Contains data, has balance
- **Frozen:** Temporarily locked
- **Closed:** Zero balance, can be garbage collected

## Rent Exemption

- Accounts must maintain minimum balance
- Prevents storage spam
- "Rent" paid to validators for storage

# Architecture Overview



# API Endpoints

## Account Management

- `POST /accounts` - Create new account record
- `GET /accounts` - List all accounts
- `GET /accounts/:id` - Get account by ID
- `GET /accounts/address/:address` - Get account by Solana address
- `PUT /accounts/:id` - Update account metadata
- `DELETE /accounts/:id` - Remove account record

## Blockchain Queries

- `GET /accounts/info/:address` - Get on-chain account info
- `GET /accounts/balance/:address` - Get SOL balance

# Database Schema

## Account Entity Fields

- `id` : Primary key (UUID)
- `address` : Solana public key (unique)
- `owner` : Account owner address
- `balance` : SOL balance (bigint)
- `isPda` : Program Derived Address flag
- `programId` : Associated program ID
- `metadata` : Additional JSON data
- `createdAt/updatedAt` : Timestamps



# Key Implementation Concepts

## PDA Generation

```
// Derive PDA from seeds and program ID
const [pdaAddress, bump] = PublicKey.findProgramAddressSync(
  [Buffer.from("escrow"), userPublicKey.toBuffer()],
  programId
);
```

## Account Info Retrieval

```
// Get account information from blockchain
const accountInfo = await connection.getAccountInfo(publicKey);
if (accountInfo) {
  // Account exists with data
  console.log('Balance:', accountInfo.lamports);
  console.log('Owner:', accountInfo.owner.toString());
}
```

# Common Patterns

## Account Creation Flow

1. Generate keypair or derive PDA
2. Check if account exists
3. Create account record in database
4. Return account information

## Balance Checking

1. Query Solana RPC for account info
2. Convert lamports to SOL
3. Cache result for performance
4. Return formatted balance

# Security Considerations

## Access Control

- Validate account ownership
- Implement proper authorization
- Rate limit API calls

## Data Validation

- Verify Solana addresses
- Sanitize metadata input
- Prevent SQL injection

# Next Steps

## Module 1 Implementation Tasks

-  Basic account CRUD operations
-  Solana RPC integration
-  PDA generation service
-  Account abstraction features

## Related Modules

- **Module 2:** Transactions & Instructions
- **Module 3:** Token Standards
- **Module 4:** Account Abstraction

# Thank You!

## Questions?

## Ready to explore Transactions & Instructions?

Next: Module 2 - Transactions & Instructions 

# **Module 2: Transactions and Instructions**

## **Building and Submitting Transactions**

# Transactions vs Instructions

## Solana Transaction Model

- **Transaction:** Container for multiple operations
- **Instructions:** Individual operations within a transaction
- **Key Difference:** One transaction can execute multiple instructions atomically

## EVM Comparison

- **EVM:** Each transaction calls one contract function
- **SVM:** Transactions can batch multiple operations

# Transaction Structure

## Core Components

- **Signatures:** Array of signer public keys
- **Message:** Contains all transaction data
  - Recent blockhash
  - Account addresses
  - Instructions array

## Instruction Format

- **Program ID:** Which program to execute
- **Accounts:** Which accounts the instruction uses
- **Data:** Instruction-specific parameters



# Transaction Lifecycle

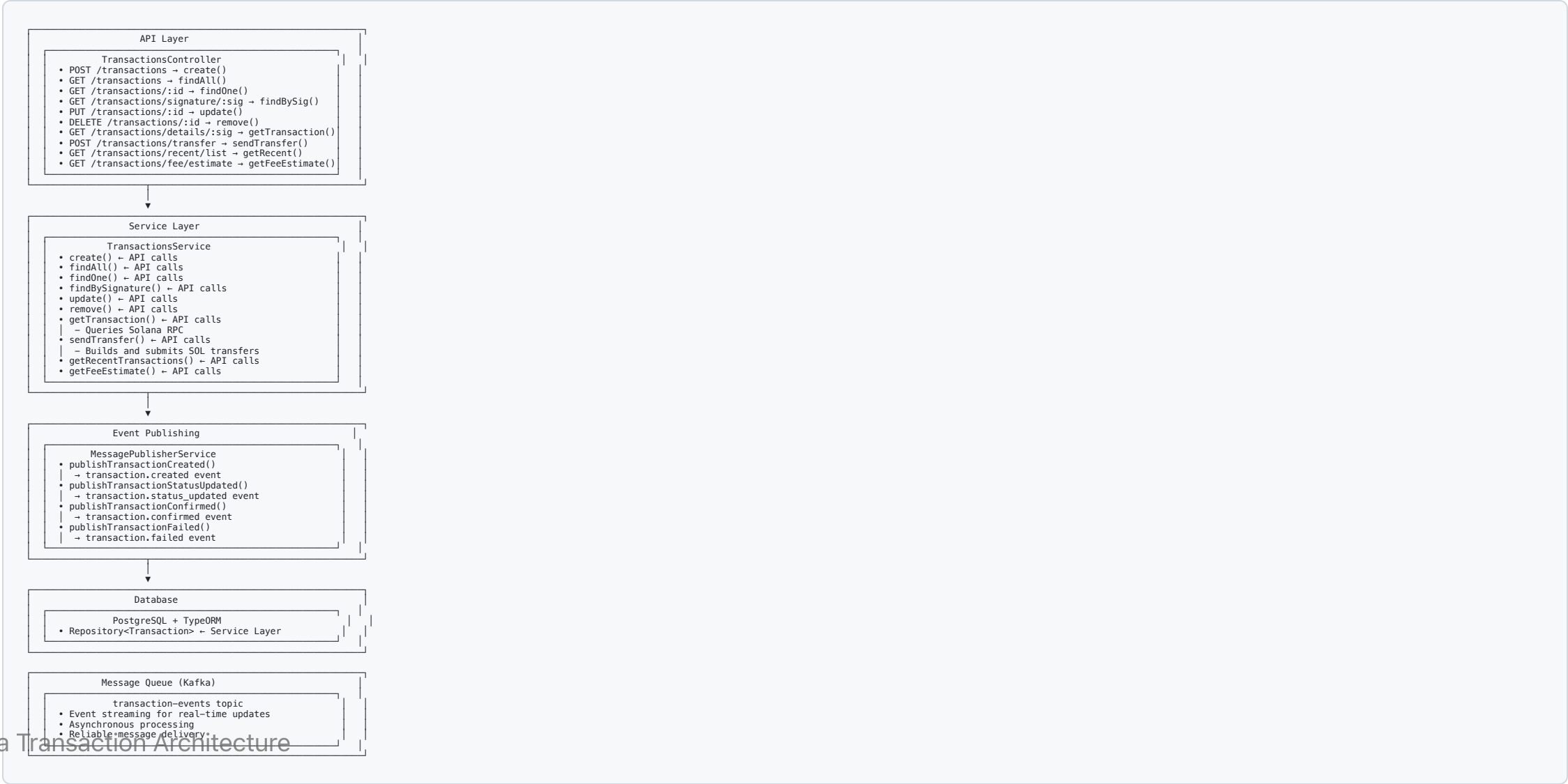
## Status Flow

1. **PENDING**: Transaction created, not yet submitted
2. **CONFIRMED**: Successfully executed on-chain
3. **FAILED**: Execution failed with error

## Key Events

- Transaction created
- Status updates
- Confirmation notifications
- Failure alerts

# Architecture Overview



## Transaction with Multiple Instructions

```
// Create multiple instructions
const instructions = [
  SystemProgram.transfer({...}),
  TokenProgram.transfer({...}),
  // ... more instructions
];

// Add to transaction
const transaction = new Transaction();
transaction.add(...instructions);

// Send transaction
const signature = await sendAndConfirmTransaction(connection, transaction, signers);
```

# Error Handling

## Common Transaction Errors

- **Insufficient Funds:** Not enough SOL for transfer + fees
- **Invalid Account:** Account doesn't exist or wrong owner
- **Program Error:** Instruction execution failed
- **Timeout:** Transaction not confirmed in time

## Retry Mechanisms

- Exponential backoff
- Fee adjustment
- Alternative RPC endpoints
- Dead letter queue for persistent failures

# Performance Considerations

## Optimization Strategies

- Batch multiple operations in single transaction
- Use efficient instruction ordering
- Implement proper fee management
- Cache frequently used data

## Monitoring

- Transaction success rates
- Confirmation times
- Fee efficiency
- Error patterns

# Security Best Practices

## Transaction Security

- Validate all input parameters
- Verify account ownership
- Implement proper authorization
- Use secure key management

## Replay Protection

- Recent blockhash prevents replay
- Unique transaction signatures
- Proper nonce handling

# Next Steps

## Module 2 Implementation Tasks

-  Basic transaction CRUD
-  SOL transfer functionality
-  Multi-instruction transactions
-  Advanced fee management
-  Token transfer transactions

## Related Modules

- **Module 1:** Accounts & Programs
- **Module 3:** Token Standards
- **Module 5:** Fee Mechanism

# Thank You!

## Questions?

### Ready to explore Token Standards?

Next: Module 3 - Token Standards 



# Module 3: Token Standards (SPL Tokens)

## Solana Program Library Tokens

# SPL vs ERC Standards

## Solana Token Standards

- **SPL (Solana Program Library):** Native token standard
- **SPL-Token:** Fungible tokens (like ERC-20)
- **SPL-Token-2022:** Enhanced token standard with extensions

## EVM Comparison

- **ERC-20:** Basic fungible tokens
- **ERC-721:** Non-fungible tokens
- **ERC-1155:** Multi-token standard

# Token Account Model

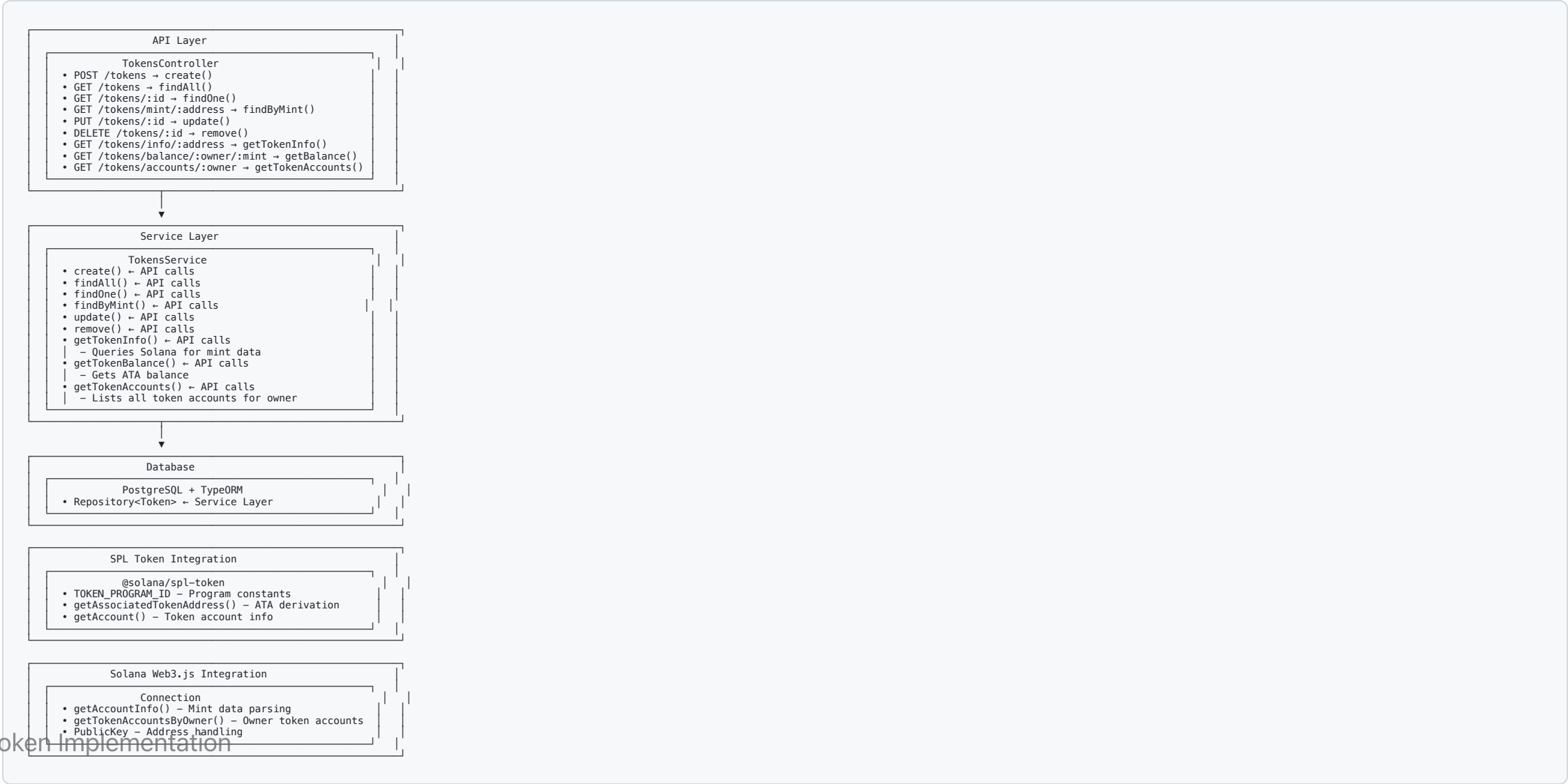
## Three Account Types

1. **Mint Account:** Token definition (supply, decimals, authorities)
2. **Token Account:** User's token balance
3. **Associated Token Account (ATA):** Deterministic token account address

## Key Differences from EVM

- **Separate Accounts:** Each token holding requires a token account
- **ATAs:** Automatic account derivation per wallet-token pair
- **No Direct Balance:** Balance stored in separate token accounts

# Architecture Overview



# Token Operations

## Creating a Token

```
// Create mint account
const mint = await createMint(
  connection,
  payer,
  mintAuthority,
  freezeAuthority,
  decimals
);

// Create token account
const tokenAccount = await getOrCreateAssociatedTokenAccount(
  connection,
  payer,
  mint,
  owner
);
```

## Token Transfer

```
// Transfer tokens between accounts
await transfer(
  connection,
  payer,
  sourceTokenAccount,
  destinationTokenAccount,
  owner,
  amount
);
```

## Minting Tokens

```
// Mint new tokens to account
await mintTo(
  connection,
  payer,
  mint,
  destinationTokenAccount,
  mintAuthority,
  amount
);
```

# SPL Token Extensions

## Token-2022 Features

- **Transfer Fees:** Charge fees on transfers
- **Confidential Transfers:** Private token amounts
- **Non-transferable Tokens:** Soul-bound tokens
- **Interest-bearing Tokens:** Automatic yield
- **Permanent Delegate:** Always-authorized delegate

## Metadata Extension

- **Token Metadata:** Name, symbol, description, image
- **Creator Verification:** Verify content creators
- **Royalties:** Automatic royalty payments



# NFT Implementation

## SPL Token NFTs

- **Supply = 1:** Single token per mint
- **Decimals = 0:** No fractional NFTs
- **Unique Metadata:** Each NFT has unique attributes

## Metaplex Standard

- **Metadata Account:** Stores NFT metadata
- **Master Edition:** Controls supply and royalties
- **Creator Verification:** Verify NFT creators

# Common Patterns

## Token Balance Checking

```
// Get token balance
const tokenAccounts = await getTokenAccountsByOwner(
  connection,
  ownerPublicKey,
  { mint: mintPublicKey }
);

const balance = tokenAccounts.value[0]?.account.data.parsed.info.tokenAmount.uiAmount;
```

## Batch Token Operations

```
// Create multiple token transfers
const instructions = [];
for (const transfer of transfers) {
  const ix = createTransferInstruction(
    transfer.source,
    transfer.destination,
    transfer.owner,
    transfer.amount
  );
  instructions.push(ix);
}

// Execute all in one transaction
const transaction = new Transaction().add(...instructions);
```

# Performance Considerations

## Optimization Strategies

- **ATA Caching:** Cache associated token addresses
- **Batch Operations:** Combine multiple transfers
- **Lazy Loading:** Load token data on demand
- **Index Optimization:** Efficient database queries

## Scalability

- **Parallel Processing:** Handle multiple token operations
- **Connection Pooling:** Efficient RPC connections
- **Rate Limiting:** Prevent API abuse

# Security Best Practices

## Token Security

- **Authority Validation:** Verify mint/freeze authorities
- **Amount Validation:** Prevent overflow/underflow
- **Account Ownership:** Verify token account ownership
- **Freeze Protection:** Handle frozen accounts

## Smart Contract Risks

- **Reentrancy:** Prevent reentrant calls
- **Access Control:** Proper permission checks
- **Input Validation:** Sanitize all inputs
- **Audit Requirements:** Security audits for custom tokens

# Next Steps

## Module 3 Implementation Tasks

-  Basic token CRUD operations
-  Token balance queries
-  Token creation/minting
-  Token transfer operations
-  NFT support
-  Token metadata management

## Related Modules

- **Module 2:** Transactions & Instructions
- **Module 4:** Account Abstraction
- **Module 10:** CPIs (Cross-Program Invocations)

# Thank You!

## Questions?

**Ready to explore Account Abstraction?**

Next: Module 4 - Account Abstraction 

# **Module 4: Account Abstraction**

## **Smart Accounts & Programmatic Control**



# What is Account Abstraction?

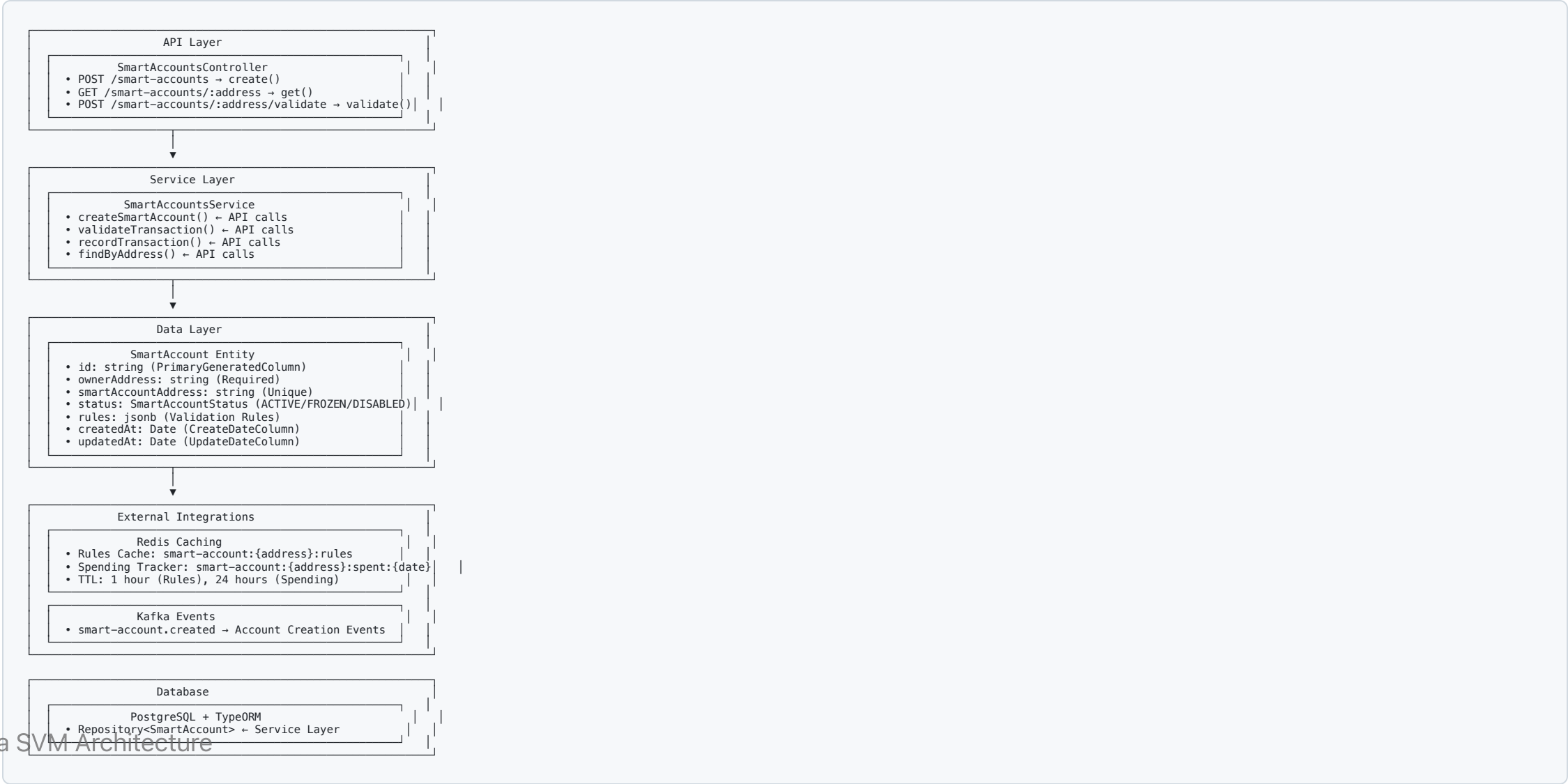
## Traditional Accounts

- **User Accounts:** Controlled by private keys
- **Program Accounts:** Immutable executable code
- **PDA:** Deterministically derived addresses

## Account Abstraction

- **Smart Accounts:** Programmatic control over account behavior
- **Validation Rules:** Custom spending limits and access controls
- **Enhanced Security:** Multi-signature and whitelist features

# Architecture Overview



# Validation Rules Structure

## Rules Object Configuration

```
{
  "maxDailySpend": 1000000,           // Daily limit in lamports
  "allowedPrograms": [                // Program ID whitelist
    "TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA",
    "111111111111111111111111111111111112"
  ],
  "requiredSigners": 2                // Multi-sig requirement
}
```

## Rule Types

- **Daily Spending Limits:** Redis-tracked budgets
- **Program Whitelisting:** Allowed program IDs only
- **Multi-signature Support:** Required number of signers

# API Endpoints

## Smart Account Management

- `POST /smart-accounts` - Create new smart account
- `GET /smart-accounts/:address` - Get smart account details
- `POST /smart-accounts/:address/validate` - Validate transaction against rules

## Validation Logic

```
validateTransaction(tx: Transaction): {valid: boolean, reason?: string} {  
  // Check account status  
  if (account.status !== 'ACTIVE') {  
    return {valid: false, reason: 'Account not active'};  
  }  
  
  // Check daily spend limit  
  const dailySpent = await redis.get(`smart-account:${address}:spent:${today}`);  
  if (dailySpent + tx.amount > rules.maxDailySpend) {  
    return {valid: false, reason: 'Daily spend limit exceeded'};  
  }  
}
```

# Smart Account Features

## Implemented Capabilities

- **Daily Spending Limits:** Redis-tracked spending budgets
- **Program Whitelisting:** Restrict interactions to approved programs
- **Multi-signature Support:** Require multiple signers for transactions
- **Account Status Control:** ACTIVE/FROZEN/DISABLED account states
- **Event-driven Architecture:** Kafka integration for monitoring

## Security Benefits

- **Reduced Attack Surface:** Limited program interactions
- **Spending Controls:** Prevent unauthorized large transfers
- **Account Recovery:** Multi-sig for lost key scenarios
- **Audit Trail:** Complete transaction history and validation logs

# PDA Integration

## Program Derived Addresses

```
// Mock implementation for development
const mockSmartAccountAddress = `smart-${ownerAddress}-${timestamp}`;

// Future: Real PDA derivation
const [smartAccountAddress, bump] = await PublicKey.findProgramAddress(
  [
    Buffer.from('smart-account'),
    new PublicKey(ownerAddress).toBuffer(),
    Buffer.from(timestamp.toString())
  ],
  programId
);
```

## PDA Benefits

- **Deterministic:** Same inputs always generate same address

# Implementation Status

## Completed Features

- Smart account entity with validation rules
- Redis caching for rules and spending tracking
- Kafka event streaming for account creation
- API endpoints for CRUD operations
- Transaction validation logic

## Future Enhancements

- Real PDA implementation with Solana Web3.js
- Multi-signature wallet integration
- Gasless transaction support
- Account recovery mechanisms

# Key Takeaways

## Account Abstraction Benefits

- **Enhanced Security:** Programmatic spending controls and whitelisting
- **User Experience:** Gasless transactions and account recovery
- **Developer Flexibility:** Custom validation rules and business logic
- **Scalability:** Off-chain validation with on-chain enforcement

## SVM Advantages

- **PDAs:** Trustless deterministic address generation
- **Parallel Processing:** High-throughput validation and execution
- **Low Fees:** Cost-effective account management
- **Composability:** Smart accounts can interact with any program



# Module 5: Fee Mechanism

## Dynamic Fee Calculation & Optimization

# Solana Fee Structure

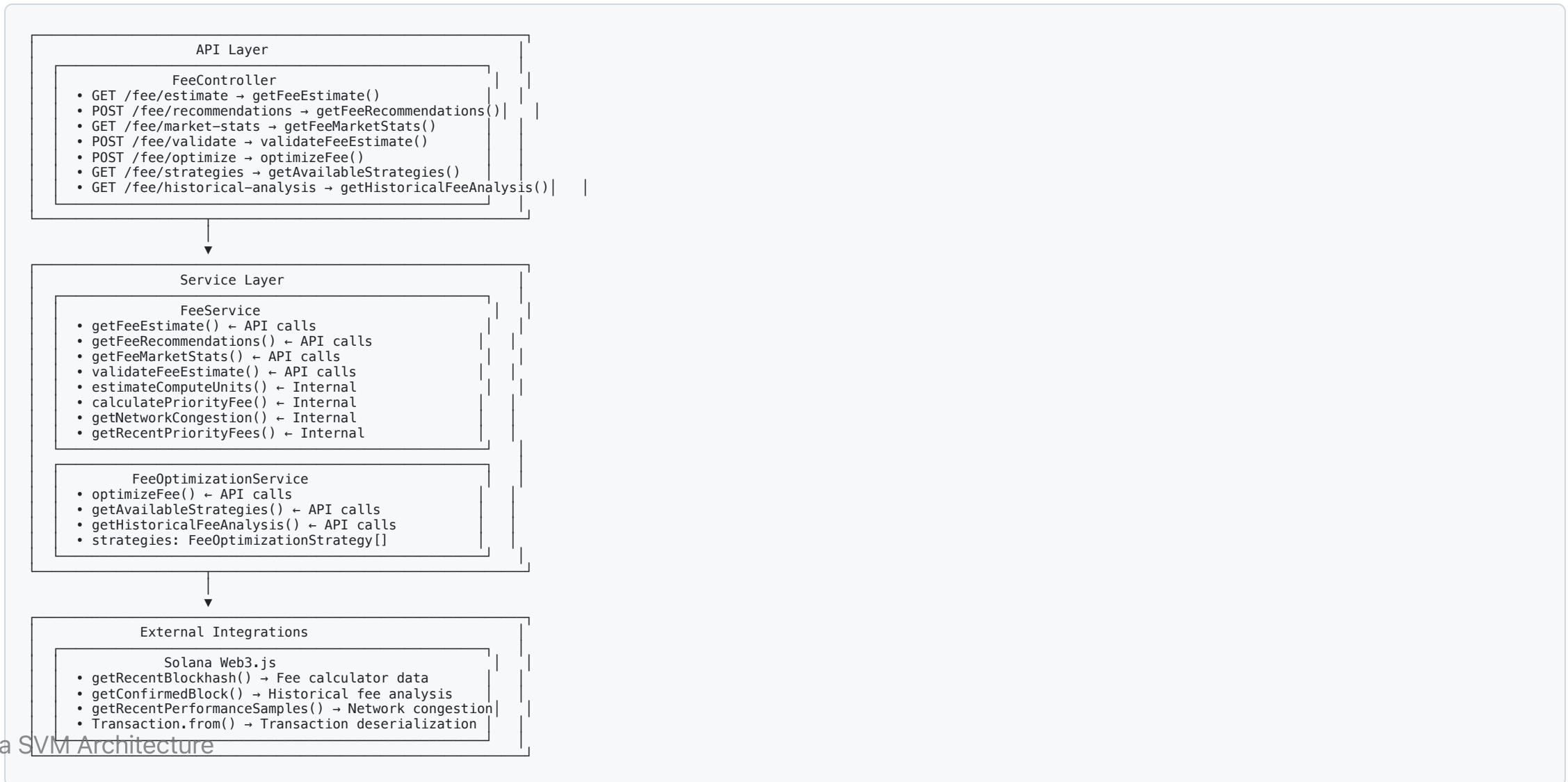
## Fee Components

- **Base Fee:** 5000 lamports per signature (fixed)
- **Priority Fee:** Variable fee for transaction ordering
- **Compute Unit Cost:** Based on program execution complexity

## Fee Market Dynamics

- **Network Congestion:** Higher fees during peak usage
- **Priority Levels:** Conservative to aggressive fee strategies
- **Historical Analysis:** Trend-based fee optimization

# Architecture Overview



# Fee Data Structures

## FeeEstimate Interface

```
interface FeeEstimate {  
  baseFee: number;           // Lamports per signature (5000)  
  priorityFee: number;       // Additional priority fee  
  totalFee: number;          // Total estimated cost  
  computeUnits: number;      // Estimated CU usage  
  feePayer: string;          // Fee payer address  
}
```

## FeeRecommendation Interface

```
interface FeeRecommendation {  
  conservative: FeeEstimate; // Low risk option  
  moderate: FeeEstimate;     // Balanced option  
  aggressive: FeeEstimate;   // High priority option  
  networkCongestion: 'low' | 'medium' | 'high';  
  recentBlockhash: string;   // Current blockhash  
}
```

# Priority Fee Levels

## Fee Multipliers

- **Min:** 0.1x multiplier (minimal priority)
- **Low:** 0.5x multiplier (reduced priority)
- **Medium:** 1.0x multiplier (standard priority)
- **High:** 2.0x multiplier (elevated priority)
- **Very High:** 5.0x multiplier (high priority)
- **Unsafe Max:** 10.0x multiplier (maximum priority)

## Strategy Selection

```
enum FeeOptimizationStrategy {  
    CONSERVATIVE = 'ConservativeStrategy', // Minimize cost, accept delays  
    BALANCED = 'BalancedStrategy', // Balance cost vs speed  
    AGGRESSIVE = 'AggressiveStrategy', // Maximize speed, higher cost
```

# Compute Unit Estimation

## Program CU Costs

```
const COMPUTE_UNIT_ESTIMATES = {  
  'SystemProgram.transfer': 5000,  
  'SystemProgram.createAccount': 10000,  
  'SPL Token operations': 8000,  
  'Other programs': 10000, // Conservative estimate  
  'Transaction overhead': 5000 // Minimum buffer  
};
```

## Estimation Logic

```
estimateComputeUnits(transaction: Transaction): number {  
  let totalCU = 5000; // Transaction overhead  
  
  for (const instruction of transaction.instructions) {  
    const programId = instruction.programId.toString();  
  
    if (programId === SystemProgram.programId.toString()) {
```

# Network Congestion Analysis

## Congestion Detection

```
getNetworkCongestion(): 'low' | 'medium' | 'high' {  
  const samples = await connection.getRecentPerformanceSamples(5);  
  const avgTPS = samples.reduce((sum, s) => sum + s.numTransactions, 0) / samples.length;  
  
  if (avgTPS > 5000) return 'high';  
  if (avgTPS > 2000) return 'medium';  
  return 'low';  
}
```

## Fee Adjustment Based on Congestion

- **Low Congestion:** Base fees only
- **Medium Congestion:** 1.5x priority fee multiplier
- **High Congestion:** 3x priority fee multiplier

# Fee Optimization Strategies

## Strategy Implementations

### Conservative Strategy

- **Goal:** Minimize cost, accept potential delays
- **Priority Fee:** 0.1x base rate
- **Success Rate:** ~60% inclusion in next block
- **Use Case:** Non-urgent transactions

### Balanced Strategy

- **Goal:** Balance cost vs speed
- **Priority Fee:** 1.0x base rate
- **Success Rate:** ~85% inclusion in next block



# Historical Fee Analysis

## Trend Analysis

```
interface HistoricalFeeAnalysis {  
  averageFee: number;           // Mean fee over period  
  feeVolatility: number;        // Standard deviation  
  bestTimes: HourlyFeeAverage[]; // Optimal transaction times  
  trend: 'increasing' | 'decreasing' | 'stable';  
}  
  
interface HourlyFeeAverage {  
  hour: number; // 0-23  
  averageFee: number;  
  successRate: number;  
}
```

## Predictive Optimization

- **Pattern Recognition:** Identify low-fee time windows

# API Endpoints

## Fee Estimation

- `GET /fee/estimate` - Get basic fee estimate for transaction
- `POST /fee/recommendations` - Get multi-level fee recommendations
- `GET /fee/market-stats` - Get current network fee statistics

## Fee Optimization

- `POST /fee/validate` - Validate a fee estimate
- `POST /fee/optimize` - Optimize fee using selected strategy
- `GET /fee/strategies` - List available optimization strategies
- `GET /fee/historical-analysis` - Get historical fee trend analysis

# Implementation Example

## Complete Fee Estimation Flow

```

async getFeeEstimate(transaction: Transaction): Promise<FeeEstimate> {
  // Estimate compute units
  const computeUnits = this.estimateComputeUnits(transaction);

  // Get network congestion
  const congestion = await this.getNetworkCongestion();

  // Calculate base fee (5000 lamports per signature)
  const signatures = transaction.signatures.length || 1;
  const baseFee = signatures * 5000;

  // Calculate priority fee based on congestion
  const priorityMultiplier = congestion === 'high' ? 3.0 :
                                congestion === 'medium' ? 1.5 : 1.0;
  const priorityFee = Math.floor(baseFee * priorityMultiplier);

  return {
    baseFee,
    priorityFee,
    totalFee: baseFee + priorityFee,
    computeUnits,
    feePayer: transaction.feePayer?.toString() || ''
  };
}

```

# Key Takeaways

## Fee Mechanism Benefits

- **Cost Efficiency:** Dynamic fee calculation prevents overpayment
- **Priority Control:** Strategic fee setting for transaction ordering
- **Network Awareness:** Congestion-based fee adjustments
- **Historical Optimization:** Data-driven fee strategy selection

## SVM Fee Advantages

- **Predictable Base Fees:** Fixed 5000 lamports per signature
- **Priority Fee Flexibility:** Variable fees for transaction ordering
- **Compute Unit Pricing:** Pay for actual resource usage
- **High Throughput:** Efficient fee market at scale

# **Module 6: Consensus and Validation**

## **Multi-Layer Security & Validation Framework**

# Validation Architecture Overview

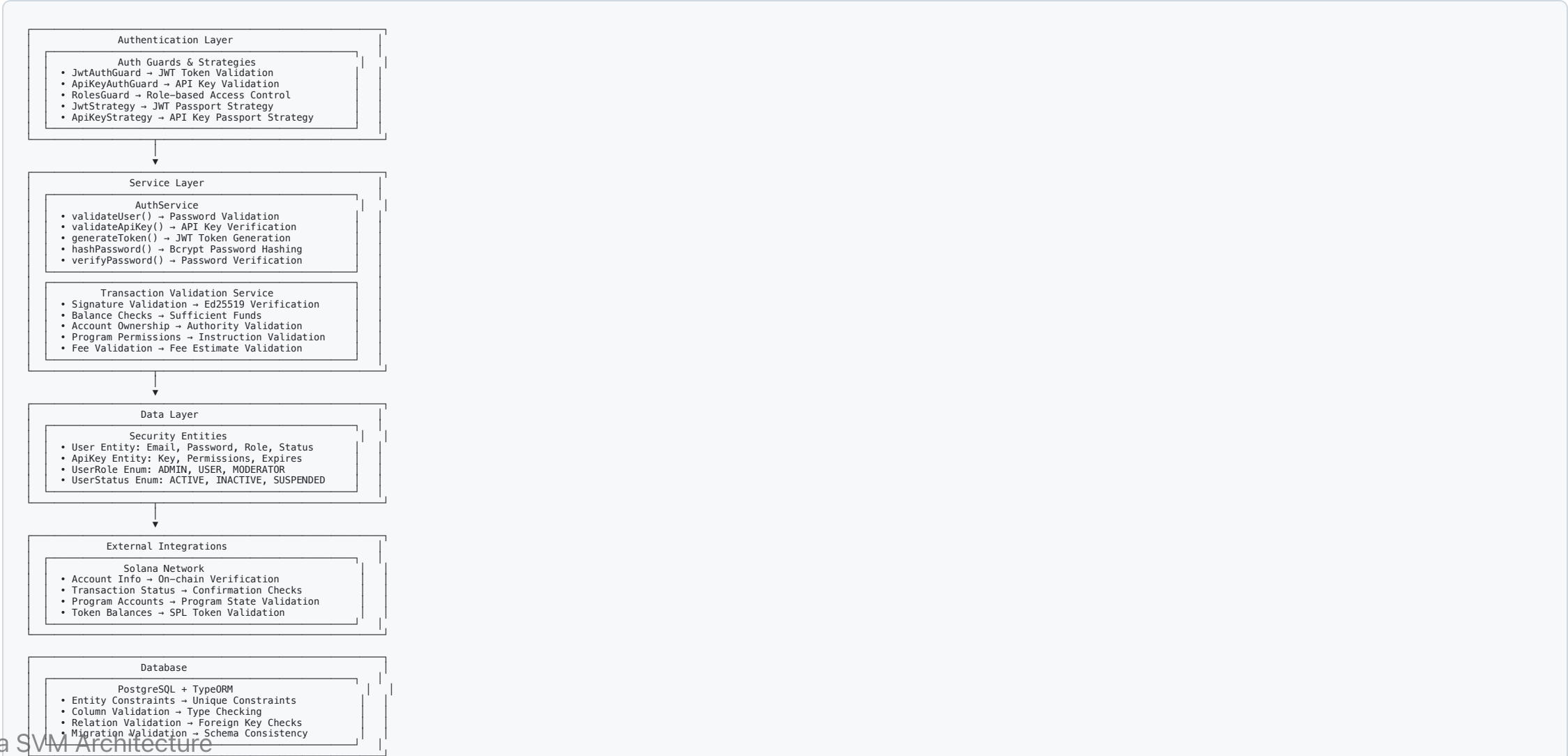
## Validation Layers

- **Authentication:** JWT tokens, API keys, role-based access
- **Input Validation:** DTO validation, data transformation
- **Business Logic:** Transaction validation, program execution
- **External Checks:** Solana network validation
- **Database:** TypeORM constraints and relationships

## Security Principles

- **Defense in Depth:** Multiple validation layers
- **Fail-Fast:** Early validation prevents unnecessary processing
- **Comprehensive Coverage:** All data paths validated
- **Error Transparency:** Clear error messages and codes

# System Architecture



# Validation Pipeline

## Request Processing Flow

1.  Auth Guards → JWT/API Key Check
2.  Input Validation → DTO/Class Validation
3.  Business Rules → Service Layer Validation
4.  External Checks → Solana Network Validation
5.  Database Ops → TypeORM Constraints
6.  Response → Validated Data

## Pipeline Benefits

- **Sequential Processing:** Each layer builds on previous validation
- **Early Failure:** Invalid requests rejected quickly
- **Resource Efficiency:** Only valid requests reach expensive operations
- **Audit Trail:** Complete validation history for debugging



# Authentication & Authorization

## Auth Guards Implementation

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const token = this.extractTokenFromHeader(request);

    if (!token) {
      throw new UnauthorizedException('No token provided');
    }

    try {
      const payload = this.jwtService.verify(token);
      request.user = payload;
      return true;
    } catch (error) {
      throw new UnauthorizedException('Invalid token');
    }
  }
}
```

# Transaction Validation

## Signature Validation

```
validateTransactionSignature(transaction: Transaction): boolean {  
  try {  
    // Verify all required signatures  
    for (const signature of transaction.signatures) {  
      const message = transaction.serializeMessage();  
      const isValid = nacl.sign.detached.verify(  
        message,  
        signature.signature,  
        signature.publicKey.toBytes()  
      );  
  
      if (!isValid) {  
        throw new BadRequestException('Invalid transaction signature');  
      }  
    }  
    return true;  
  } catch (error) {  
    throw new BadRequestException('Transaction signature validation failed');  
  }  
}
```

# Program Execution Validation

## SVM Runtime Validation

```
validateProgramExecution(programId: PublicKey, instruction: TransactionInstruction): void {  
    // Validate program exists and is executable  
    const accountInfo = await this.connection.getAccountInfo(programId);  
    if (!accountInfo) {  
        throw new NotFoundException('Program account not found');  
    }  
  
    if (!accountInfo.executable) {  
        throw new BadRequestException('Account is not executable');  
    }  
  
    // Check compute unit limits  
    const estimatedCU = this.estimateComputeUnits(instruction);  
    if (estimatedCU > MAX_COMPUTE_UNITS) {  
        throw new BadRequestException('Instruction exceeds compute unit limit');  
    }  
  
    // Validate instruction data format  
    this.validateInstructionData(instruction);  
}
```

# Smart Account Validation

## Rule-Based Validation

```

async validateSmartAccountTransaction(
  smartAccount: SmartAccount,
  transaction: Transaction
): Promise<ValidationResult> {

  // Check account status
  if (smartAccount.status !== SmartAccountStatus.ACTIVE) {
    return { valid: false, reason: 'Smart account is not active' };
  }

  // Validate daily spending limit
  const dailySpent = await this.getDailySpending(smartAccount.address);
  const transactionAmount = this.calculateTransactionAmount(transaction);

  if (dailySpent + transactionAmount > smartAccount.rules.maxDailySpend) {
    return { valid: false, reason: 'Daily spending limit exceeded' };
  }

  // Check program whitelist
  const allowedPrograms = new Set(smartAccount.rules.allowedPrograms);
  for (const ix of transaction.instructions) {
    if (!allowedPrograms.has(ix.programId.toString())) {
      return { valid: false, reason: 'Program not in whitelist' };
    }
  }
}

```

# Input Validation & DTOs

## Class Validator Integration

```
import { IsNotEmpty, IsEmail, IsString, MinLength } from 'class-validator';

export class CreateUserDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  @IsString()
  @MinLength(8)
  password: string;

  @IsNotEmpty()
  @IsString()
  role: UserRole;
}

export class CreateTransactionDto {
  @IsNotEmpty()
  @IsString()
  fromAddress: string;

  @IsNotEmpty()
  @IsString()
  toAddress: string;

  @IsNotEmpty()
  @Min(0)
  amount: number;

  @IsOptional()
  fee: string;
}
```

# Error Handling & Responses

## Standardized Error Responses

```
export class ValidationErrorResponse {  
  statusCode: number;  
  message: string | string[];  
  error: string;  
  timestamp: string;  
  path: string;  
}  
  
// Example error responses  
const errors = {  
  400: { statusCode: 400, message: 'Validation failed', error: 'Bad Request' },  
  401: { statusCode: 401, message: 'Unauthorized', error: 'Unauthorized' },  
  403: { statusCode: 403, message: 'Forbidden', error: 'Forbidden' },  
  404: { statusCode: 404, message: 'Not found', error: 'Not Found' },  
  409: { statusCode: 409, message: 'Conflict', error: 'Conflict' }  
};
```

# Database Validation

## TypeORM Entity Validation

```
@Entity()
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  @Index()
  email: string;

  @Column()
  passwordHash: string;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;

  @Column({
    type: 'enum',
    enum: UserStatus,
    default: UserStatus.ACTIVE
  })
  status: UserStatus;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
```

# Key Takeaways

## Validation Framework Benefits

- **Multi-Layer Security:** Authentication, authorization, and business rules
- **Comprehensive Coverage:** All data inputs and operations validated
- **Performance Optimized:** Early validation prevents unnecessary processing
- **Developer Experience:** Clear error messages and validation feedback

## SVM Consensus Advantages

- **Cryptographic Security:** Ed25519 signature validation
- **Parallel Validation:** High-throughput transaction processing
- **State Consistency:** Atomic validation across all operations
- **Network Security:** On-chain verification of all state changes



# Module 7: Signing and Cryptography

## Ed25519 Digital Signatures & Key Management

# Cryptographic Foundations

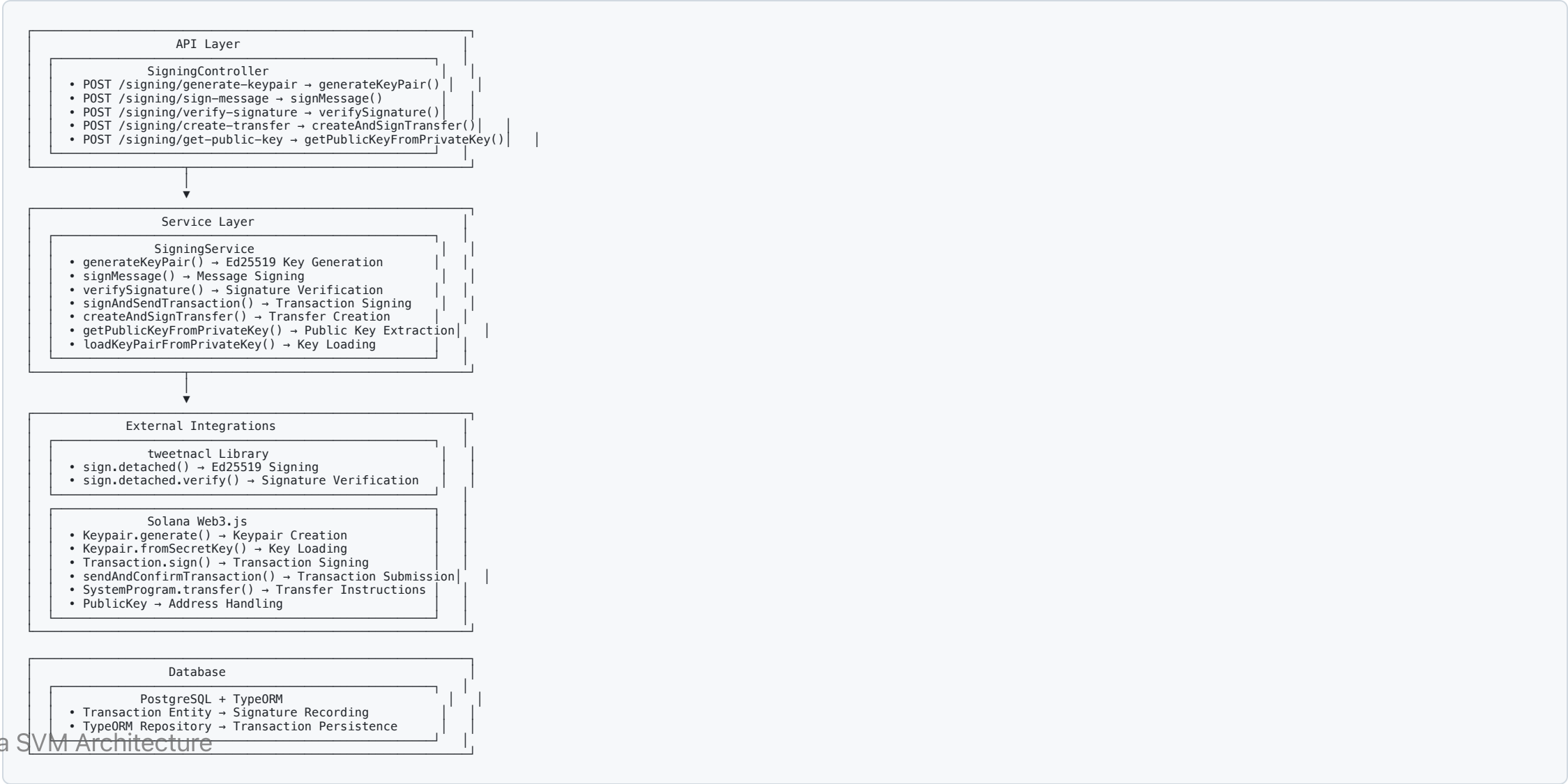
## Ed25519 Digital Signatures

- **Algorithm:** Ed25519 elliptic curve cryptography
- **Key Size:** 256-bit keys (32 bytes)
- **Signature Size:** 512-bit signatures (64 bytes)
- **Security:** Post-quantum resistant design
- **Performance:** Fast signing and verification

## Solana's Crypto Requirements

- **Transaction Signing:** All transactions must be signed
- **Account Security:** Private keys control account ownership
- **Program Security:** Signed deployments and upgrades
- **Multi-signature:** Multiple key validation

# Architecture Overview



# Key Management

## Key Generation

```
generateKeyPair(): KeyPairResponse {  
  // Generate new Ed25519 keypair  
  const keypair = Keypair.generate();  
  
  // Return only public key (security best practice)  
  return {  
    publicKey: keypair.publicKey.toString(),  
    // private key is NEVER returned via API  
  };  
}
```

## Private Key Handling

```
loadKeyPairFromPrivateKey(privateKeyBase64: string): Keypair {  
  try {  
    // Decode base64 private key
```

# Message Signing

## Signing Workflow

```
async signMessage(signMessageDto: SignMessageDto): Promise<SigningResult> {  
  try {  
    // Load keypair from private key  
    const keypair = this.loadKeyPairFromPrivateKey(signMessageDto.privateKey);  
  
    // Decode message from base64  
    const messageBytes = Buffer.from(signMessageDto.message, 'base64');  
  
    // Sign message using Ed25519  
    const signature = nacl.sign.detached(messageBytes, keypair.secretKey);  
  
    // Return base64 encoded signature  
    return {  
      signature: Buffer.from(signature).toString('base64'),  
      publicKey: keypair.publicKey.toString(),  
      success: true  
    };  
  } catch (error) {  
    throw new BadRequestException('Message signing failed');  
  }  
}
```

# Signature Verification

## Verification Process

```
async verifySignature(verifyDto: VerifySignatureDto): Promise<VerificationResult> {  
  try {  
    // Decode inputs from base64  
    const messageBytes = Buffer.from(verifyDto.message, 'base64');  
    const signatureBytes = Buffer.from(verifyDto.signature, 'base64');  
    const publicKey = new PublicKey(verifyDto.publicKey);  
  
    // Verify signature using Ed25519  
    const isValid = nacl.sign.detached.verify(  
      messageBytes,  
      signatureBytes,  
      publicKey.toBytes()  
    );  
  
    return {  
      isValid,  
      publicKey: publicKey.toString(),  
      message: isValid ? 'Signature is valid' : 'Signature is invalid'  
    };  
  } catch (error) {  
    throw new BadRequestException('Signature verification failed');  
  }  
}
```

# Transaction Signing

## Complete Transaction Flow

```

async createAndSignTransfer(transferDto: CreateTransferDto): Promise<TransactionResult> {
  try {
    // Load sender's keypair
    const senderKeypair = this.loadKeyPairFromPrivateKey(transferDto.privateKey);

    // Create transfer instruction
    const transferInstruction = SystemProgram.transfer({
      fromPubkey: senderKeypair.publicKey,
      toPubkey: new PublicKey(transferDto.toAddress),
      lamports: transferDto.amount
    });

    // Build transaction
    const transaction = new Transaction().add(transferInstruction);

    // Get recent blockhash
    const { blockhash } = await this.connection.getRecentBlockhash();
    transaction.recentBlockhash = blockhash;
    transaction.feePayer = senderKeypair.publicKey;

    // Sign transaction
    transaction.sign(senderKeypair);

    // Submit to network
    const signature = await this.connection.sendAndConfirmTransaction(transaction);

    // Record transaction in database
    await this.recordTransaction(signature, transferDto);

    return {
      signature,
      success: true,
      transactionId: signature
    };
  } catch (error) {

```

# Transaction Workflow

## Signing Workflow Steps

1.  Load Keypair → Private Key to Keypair
2.  Build Transaction → Instruction Assembly
3.  Sign Transaction → Ed25519 Signing
4.  Submit to Network → `sendAndConfirmTransaction`
5.  Record Transaction → Database Persistence

## Transaction Components

- **Instructions:** Program calls and data
- **Recent Blockhash:** Prevents replay attacks
- **Fee Payer:** Account paying transaction fees
- **Signatures:** Ed25519 signatures from required signers



# API Endpoints

## Key Management

- `POST /signing/generate-keypair` - Generate new Ed25519 keypair
- `POST /signing/get-public-key` - Extract public key from private key

## Message Signing

- `POST /signing/sign-message` - Sign arbitrary message
- `POST /signing/verify-signature` - Verify message signature

## Transaction Operations

- `POST /signing/create-transfer` - Create and sign SOL transfer

## Response Formats

# Security Considerations

## Implemented Security Measures

- **Private Key Protection:** Never returned via API responses
- **Input Validation:** Strict format checking for all inputs
- **Error Sanitization:** No sensitive data in error messages
- **Base64 Encoding:** Safe transport of binary cryptographic data
- **Transaction Recording:** Complete audit trail

## Cryptographic Best Practices

- **Ed25519 Standard:** Industry-standard elliptic curve cryptography
- **Deterministic Signatures:** Consistent signature generation
- **Replay Protection:** Blockhash prevents transaction replay
- **Multi-signature Ready:** Supports multiple signer validation

# MPC Integration (Future)

## Multi-Party Computation Features

- **Key Share Distribution:** Split private keys across multiple parties
- **Threshold Signing:** Require minimum signatures for validity
- **Secure Reconstruction:** Combine shares without exposing full key
- **Byzantine Fault Tolerance:** Resilient to malicious participants

## MPC Benefits

- **Enhanced Security:** No single point of key compromise
- **Distributed Trust:** Multiple parties required for signing
- **Fault Tolerance:** System continues with some parties offline
- **Regulatory Compliance:** Addresses custody and control requirements

# Key Takeaways

## Cryptography Implementation

- **Ed25519 Focus:** Solana's native cryptographic algorithm
- **Secure Key Management:** Private keys never exposed via API
- **Complete Transaction Flow:** From key generation to network submission
- **Audit Trail:** All operations recorded in database

## SVM Cryptographic Advantages

- **High Performance:** Fast Ed25519 operations at scale
- **Parallel Processing:** Multiple signature verifications simultaneously
- **Network Security:** Cryptographic validation of all state changes
- **Future MPC Ready:** Foundation for advanced cryptographic schemes

# Module 8: Multi-Party Computation (MPC)

## Threshold Cryptography for Secure Signing

# What is MPC?

## Multi-Party Computation

- **Distributed Cryptography:** Multiple parties compute together without revealing secrets
- **Threshold Security:**  $t$ -of- $n$  scheme (need  $t$  shares out of  $n$  total to sign)
- **Secure Signing:** Sign transactions without exposing private keys

## Use Cases

- **Multi-sig Wallets:** Distributed control over funds
- **Secure Custody:** Institutional-grade key management
- **Decentralized Signing:** No single point of failure

# Threshold Schemes

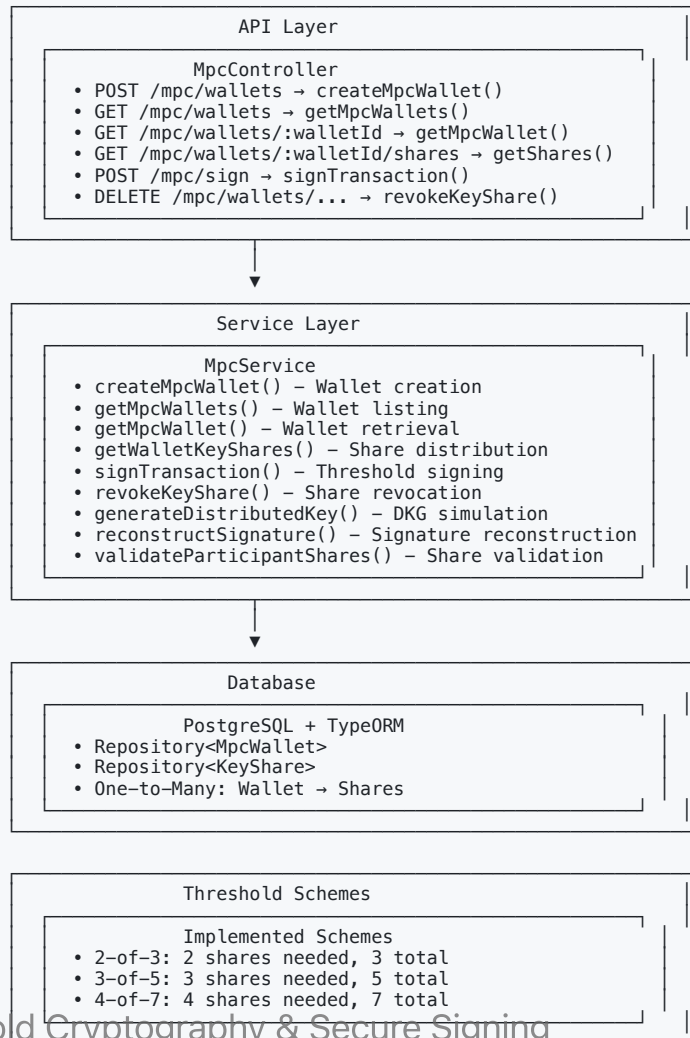
## Common Configurations

- **2-of-3**: 2 shares needed, 3 total shares
- **3-of-5**: 3 shares needed, 5 total shares
- **4-of-7**: 4 shares needed, 7 total shares

## Security Properties

- **t shares**: Minimum required for signing
- **n shares**: Total shares distributed
- **No single point**: No individual can sign alone
- **Flexible recovery**: Replace compromised shares

# Architecture Overview





## Threshold Signing

```
// Sign transaction with 2 shares (for 2-of-3 wallet)
const signature = await mpcService.signTransaction({
  walletId: wallet.id,
  transaction: serializedTransaction,
  shares: [
    { participantId: "alice", shareIndex: 0, encryptedShare: aliceShare },
    { participantId: "bob", shareIndex: 1, encryptedShare: bobShare }
  ]
});

// Use signature in Solana transaction
const tx = Transaction.from(Buffer.from(serializedTransaction));
tx.addSignature(wallet.publicKey, signature);
```

# Error Handling

## Common Errors

- **BadRequestException**: Invalid parameters or insufficient shares
- **NotFoundException**: Wallet or share not found
- **UnauthorizedException**: Invalid participant authentication
- **ForbiddenException**: Threshold not met for signing

## Validation Checks

- Participant public key verification
- Share encryption validation
- Threshold requirement enforcement
- **Wallet status verification**

## Use Cases & Benefits

### Institutional Custody

- **Secure Key Management:** No single employee can access funds
- **Regulatory Compliance:** Audit trails and access controls
- **Business Continuity:** Share recovery for lost access

### Decentralized Applications

- **DAO Treasury:** Multi-sig control for community funds
- **DeFi Protocols:** Secure oracle or bridge operations
- **NFT Marketplaces:** Escrow and royalty management

# Performance Considerations

## Optimization Strategies

- **Share Caching:** Cache decrypted shares for session
- **Batch Signing:** Process multiple transactions together
- **Connection Pooling:** Efficient database connections
- **Async Processing:** Non-blocking cryptographic operations

## Scalability

- **Horizontal Scaling:** Multiple MPC service instances
- **Load Balancing:** Distribute signing requests
- **Rate Limiting:** Prevent abuse and DoS attacks

# Security Best Practices

## Operational Security

- **Secure Share Storage:** Encrypted cold storage for shares
- **Access Logging:** Complete audit trail of all operations
- **Regular Rotation:** Periodic key share rotation
- **Backup Procedures:** Secure backup and recovery processes

## Cryptographic Security

- **Key Share Encryption:** Strong encryption for stored shares
- **Secure Communication:** TLS for all API communications
- **Participant Verification:** Multi-factor authentication
- **Compromise Response:** Immediate share revocation procedures

# Integration with Solana

## Transaction Signing

- **Ed25519 Compatibility:** Works with Solana's signature scheme
- **Transaction Serialization:** Standard Solana transaction format
- **Fee Management:** Integrated with fee estimation services

## Wallet Abstraction

- **Unified Interface:** Same API as regular wallets
- **Transparent Operation:** Applications don't need MPC awareness
- **Fallback Support:** Regular keypair signing as backup

# Next Steps

## Module 8 Implementation Tasks

-  Threshold signature generation
-  Distributed key generation
-  Key share management
-  Signature reconstruction
-  MPC wallet creation
-  Secure key distribution
-  MPC transaction signing
-  Recovery mechanisms

## Related Modules

Threshold Cryptography & Secure Signing

- **Module 7: Signing & Cryptography**

# Thank You!

## Questions?

**MPC enables secure, distributed control over blockchain assets!**

[Back to Study Topics](#) 



# **Module 9: Solana Virtual Machine (SVM)**

## **Runtime Architecture and Program Execution**

# What is the SVM?

## Solana Virtual Machine

- **Sealevel Runtime:** Solana's parallel smart contract execution engine
- **BPF Programs:** Berkeley Packet Filter bytecode for programs
- **Parallel Processing:** Execute multiple transactions simultaneously
- **Resource Management:** Compute units and gas metering

## Key Differences from EVM

- **Parallel Execution:** Multiple transactions can run at once
- **Account-Based:** Programs and data stored in accounts
- **No Gas Wars:** Priority fees instead of gas auctions

# SVM Architecture

## Core Components

- **Programs:** Executable bytecode stored in accounts
- **Runtime:** Sealevel execution environment
- **Compute Units:** Resource allocation and metering
- **Parallel Processing:** Concurrent transaction execution

## Execution Model

- **Single-threaded Accounts:** No race conditions
- **Cross-program Calls:** Programs can invoke other programs
- **State Consistency:** Atomic transaction execution

# Architecture Overview



## Executing a Program

```
// Execute single instruction
const execution = await svmService.executeProgram({
  programId: program.programId,
  instruction: {
    accounts: [/* account metas */],
    data: instructionData,
    programId: program.programId
  },
  signers: [payerKeypair],
  computeUnitLimit: 100000
});

console.log('Execution signature:', execution.transactionId);
console.log('Compute units used:', execution.computeUnitsUsed);
```

## Parallel Execution

```
// Execute multiple instructions in parallel
const results = await svmService.executeParallel({
  executions: [
    { programId: prog1, instruction: instr1 },
    { programId: prog2, instruction: instr2 },
    { programId: prog3, instruction: instr3 }
  ],
  continueOnFailure: true, // Continue if one fails
  maxConcurrent: 5 // Limit concurrency
});

// Check results
results.forEach((result, index) => {
  if (result.status === 'fulfilled') {
    console.log(`Execution ${index} succeeded:`, result.value.transactionId);
  } else {
    console.log(`Execution ${index} failed:`, result.reason);
  }
});
```

# Compute Units (CU) Management

## CU Concepts

- **Compute Units:** Measure of computational work
- **CU Price:** Priority fee per CU
- **CU Limit:** Maximum allowed per transaction
- **CU Budget:** Total available for transaction

## Setting CU Limits

```
// Set compute unit limit and price
const modifyComputeUnits = ComputeBudgetProgram.setComputeUnitLimit({
  units: 100000
});

const addPriorityFee = ComputeBudgetProgram.setComputeUnitPrice({
  microLamports: 5000 // 0.000005 SOL per CU
});
```

# Metrics and Monitoring

## Execution Analytics

- **Performance Tracking:** Execution time measurement
- **Resource Usage:** CU consumption monitoring
- **Success Rates:** Execution outcome statistics
- **Cost Analysis:** Gas expenditure tracking
- **Trend Analysis:** Historical performance patterns

## Monitoring Endpoints

- `GET /svm/metrics/executions` - Get execution metrics
- `GET /svm/programs/:programId/stats` - Get program statistics
- `GET /svm/runtime/info` - Get runtime information



# Security and Validation

## Execution Security

- **Program Verification:** Bytecode validation before execution
- **Access Control:** Owner permission checks
- **Gas Limits:** Resource bound enforcement
- **Account Validation:** Address verification
- **Error Isolation:** Failure containment

## Validation Checks

- Program ownership verification
- Account existence validation
- Instruction data sanitization
- Resource limit enforcement

# Performance Optimization

## Optimization Strategies

- **Parallel Execution:** Run independent operations concurrently
- **CU Optimization:** Right-size compute unit limits
- **Batch Processing:** Combine related operations
- **Caching:** Cache program and account data
- **Connection Pooling:** Efficient RPC connections

## Scalability Features

- **Horizontal Scaling:** Multiple SVM service instances
- **Load Balancing:** Distribute execution requests
- **Async Processing:** Non-blocking execution
- **Resource Monitoring:** Track and limit resource usage

# Integration with Solana

## Web3.js Integration

- **PublicKey:** Address handling and validation
- **Transaction:** Transaction building and serialization
- **SystemProgram:** Account creation and management
- **ComputeBudgetProgram:** CU limit and priority setting
- **sendAndConfirmTransaction:** Transaction submission and confirmation

## Network Support

- **Local Validator:** Development and testing
- **Devnet/Testnet:** Staging environments
- **Mainnet:** Production deployment
- **RPC Rotation:** Failover and load balancing

# Next Steps

## Module 9 Implementation Tasks

-  Program entity and CRUD operations
-  Program deployment functionality
-  Single program execution
-  Parallel transaction execution
-  Gas metering and tracking
-  SVM runtime monitoring
-  Local test validator setup
-  Multi-network support

## Related Modules

SVM Runtime & Program Execution

- **Module 10:** Cross-Program Invocations

# Thank You!

## Questions?

**SVM enables parallel smart contract execution on Solana!**

[Back to Study Topics](#) 

# Module 10: Cross-Program Invocations (CPIs)

## Program Composition & Interoperability

# What are Cross-Program Invocations?

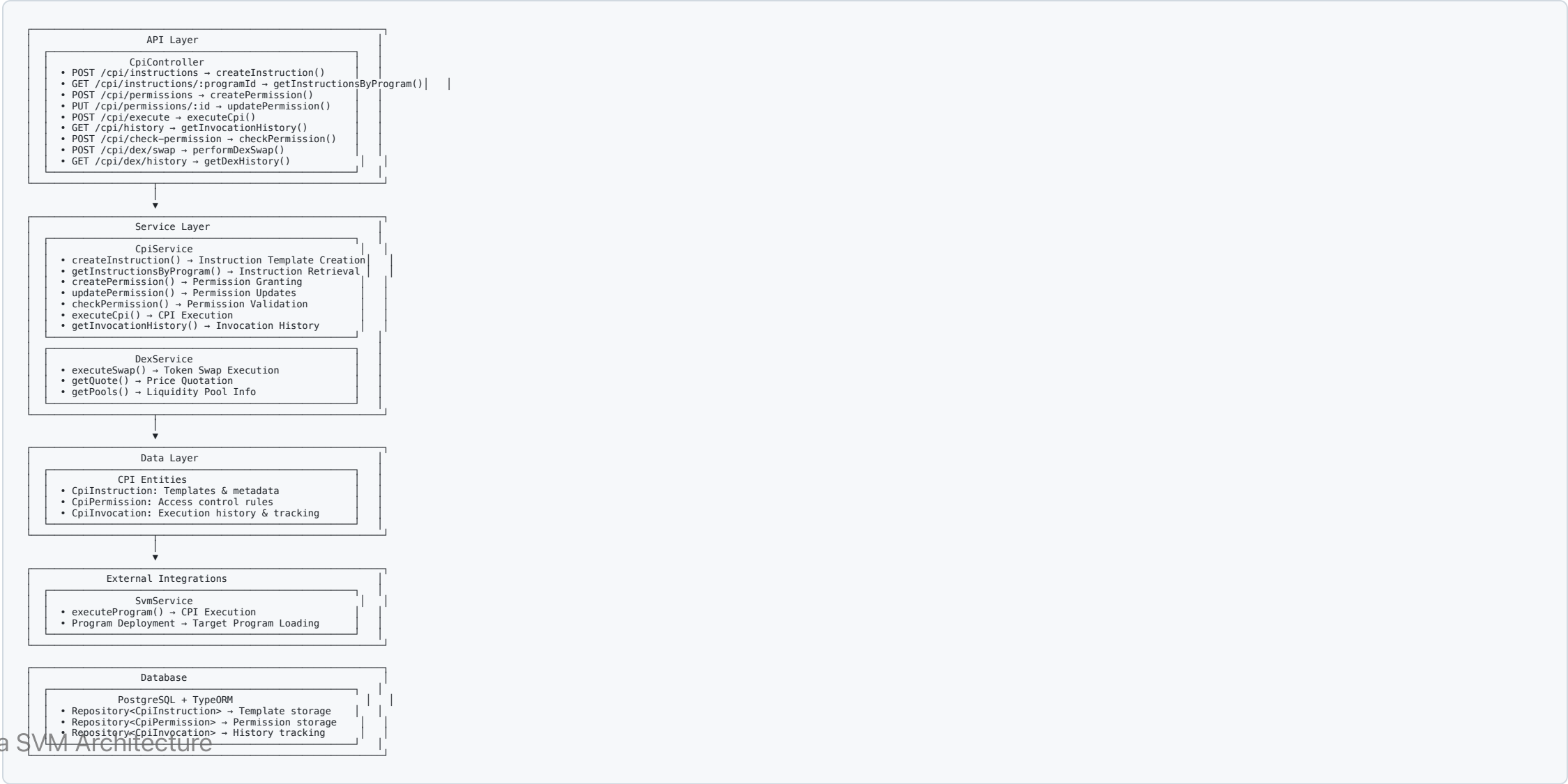
## CPI Fundamentals

- **Program Calls:** One program invoking another program's instructions
- **Composability:** Building complex functionality from simple programs
- **Trustless Execution:** Programs execute with full SVM runtime security
- **Atomic Operations:** All CPI calls succeed or fail together

## SVM Advantages

- **Single Transaction:** Multiple program interactions in one atomic operation
- **Shared State:** Programs can read and modify shared accounts
- **Gas Efficiency:** No additional transaction overhead for CPI calls
- **Security Isolation:** Each program maintains its own security boundaries

# Architecture Overview





# CPI Permission System

## Permission Management

```
interface CpiPermission {  
  id: string;  
  programId: string;           // Target program being granted access  
  granterProgramId: string;    // Program granting the permission  
  permissionType: 'read' | 'write' | 'execute';  
  accountId?: string;          // Specific account (optional)  
  expiresAt?: Date;            // Permission expiration  
  isActive: boolean;  
}
```

## Permission Validation

```
async checkPermission(  
  callerProgramId: string,  
  targetProgramId: string,  
  permissionType: string,  
  ...args: any[]  
) {  
  // Validation logic  
}
```

# Instruction Templates

## CPI Instruction Structure

```
interface CpiInstruction {  
    id: string;  
    programId: string;           // Target program ID  
    callerProgramId: string;     // Calling program ID  
    instructionData: any;        // Instruction payload  
    accounts: AccountMeta[];     // Account metadata array  
    methodName: string;          // Method identifier  
    requiresPermission: boolean; // Permission check required  
    permissionLevel: string;      // Required permission type  
    isActive: boolean;  
}  
  
interface AccountMeta {  
    pubkey: PublicKey;  
    isSigner: boolean;  
    isWritable: boolean;  
}
```

# CPI Execution Flow

## Execution Process

```

async executeCpi(executionDto: ExecuteCpiDto): Promise<CpiInvocation> {
  // 1. Permission Check
  if (executionDto.requiresPermission) {
    const hasPermission = await this.checkPermission(
      executionDto.callerProgramId,
      executionDto.targetProgramId,
      executionDto.permissionType
    );

    if (!hasPermission) {
      throw new ForbiddenException('Insufficient CPI permissions');
    }
  }

  // 2. Create invocation record
  const invocation = await this.createInvocationRecord(executionDto);

  try {
    // 3. Execute via SVM service
    const result = await this.svmService.executeProgram({
      programId: executionDto.targetProgramId,
      instruction: executionDto.instruction,
      accounts: executionDto.accounts
    });

    // 4. Update invocation with success
    invocation.status = 'success';
    invocation.transactionId = result.signature;
    invocation.gasUsed = result.computeUnits;
  } catch (error) {
    // 5. Record failure
    invocation.status = 'failed';
    invocation.errorMessage = error.message;
  }
}

```

# DEX Operations

## Token Swap Implementation

```
async performDexSwap(swapDto: DexSwapDto): Promise<SwapResult> {  
  // Get quote from DEX  
  const quote = await this.getQuote({  
    inputMint: swapDto.inputMint,  
    outputMint: swapDto.outputMint,  
    amount: swapDto.amount,  
    slippage: swapDto.slippage  
  });  
  
  // Check slippage tolerance  
  if (quote.priceImpact > swapDto.maxSlippage) {  
    throw new BadRequestException('Price impact too high');  
  }  
  
  // Execute swap via CPI  
  const swapInstruction = await this.createSwapInstruction(quote);  
  
  const result = await this.cpiService.executeCpi({  
    callerProgramId: this.dexProgramId,  
    targetProgramId: quote.ammProgramId,  
    instruction: swapInstruction,  
    accounts: quote.accounts,  
    requiresPermission: true,  
    permissionType: 'execute'  
  });  
  
  return {  
    signature: result.transactionId,  
    outputAmount: quote.outputAmount,  
    priceImpact: quote.priceImpact  
  };  
}
```

# Common CPI Patterns

## Token Operations

```
// SPL Token Transfer via CPI
const tokenTransferIx = Token.createTransferInstruction(
  TOKEN_PROGRAM_ID,
  sourceTokenAccount,
  destinationTokenAccount,
  owner,
  [],
  amount
);

// Execute via CPI
await cpiService.executeCpi({
  callerProgramId: myProgramId,
  targetProgramId: TOKEN_PROGRAM_ID,
  instruction: tokenTransferIx,
  accounts: [
    { pubkey: sourceTokenAccount, isSigner: false, isWritable: true },
    { pubkey: destinationTokenAccount, isSigner: false, isWritable: true },
    { pubkey: owner, isSigner: true, isWritable: false }
  ]
});
```

# Security Features

## CPI Security Measures

- **Permission Validation:** Granular access control enforcement
- **Account Verification:** Address validation and ownership checks
- **Execution Isolation:** Failure containment within CPI boundaries
- **Audit Trail:** Complete invocation logging and tracking
- **Gas Metering:** Resource usage monitoring and limits

## Error Handling

```
class CpiExecutionError extends Error {  
  constructor(  
    public readonly invocationId: string,  
    public readonly errorCode: string,  
    public readonly details: any  
  ) {}  
}
```

# API Endpoints

## Instruction Management

- `POST /cpi/instructions` - Create CPI instruction template
- `GET /cpi/instructions/:programId` - Get instructions for program

## Permission Management

- `POST /cpi/permissions` - Create CPI permission
- `PUT /cpi/permissions/:id` - Update permission
- `POST /cpi/check-permission` - Validate permission

## Execution & History

- `POST /cpi/execute` - Execute CPI call
- `GET /cpi/history` - Get invocation history

# Advanced CPI Patterns

## Program Derived Address Signing

```
// CPI with PDA signing
const [pdaAddress, bump] = await PublicKey.findProgramAddress(
  [Buffer.from('authority')],
  programId
);

const instruction = new TransactionInstruction({
  keys: [
    { pubkey: pdaAddress, isSigner: false, isWritable: true },
    // ... other accounts
  ],
  programId: targetProgramId,
  data: instructionData
});

// Execute with PDA authority
await cpiService.executeCpi({
  callerProgramId: programId,
  targetProgramId: targetProgramId,
  instruction,
  accounts: instruction.keys,
```



# Key Takeaways

## CPI Architecture Benefits

- **Program Composability:** Build complex protocols from simple programs
- **Atomic Execution:** All CPI calls succeed or fail together
- **Security Isolation:** Each program maintains its own security model
- **Performance Efficiency:** No additional transaction overhead

## SVM CPI Advantages

- **High Throughput:** Parallel CPI execution across programs
- **Shared State:** Programs can securely share and modify accounts
- **Gas Optimization:** Efficient resource usage for complex operations
- **Trustless Composition:** Programs can interact without trusted intermediaries

# **Module 11: Events and Logging**

## **Real-Time Monitoring & Event Streaming**

# Event-Driven Architecture

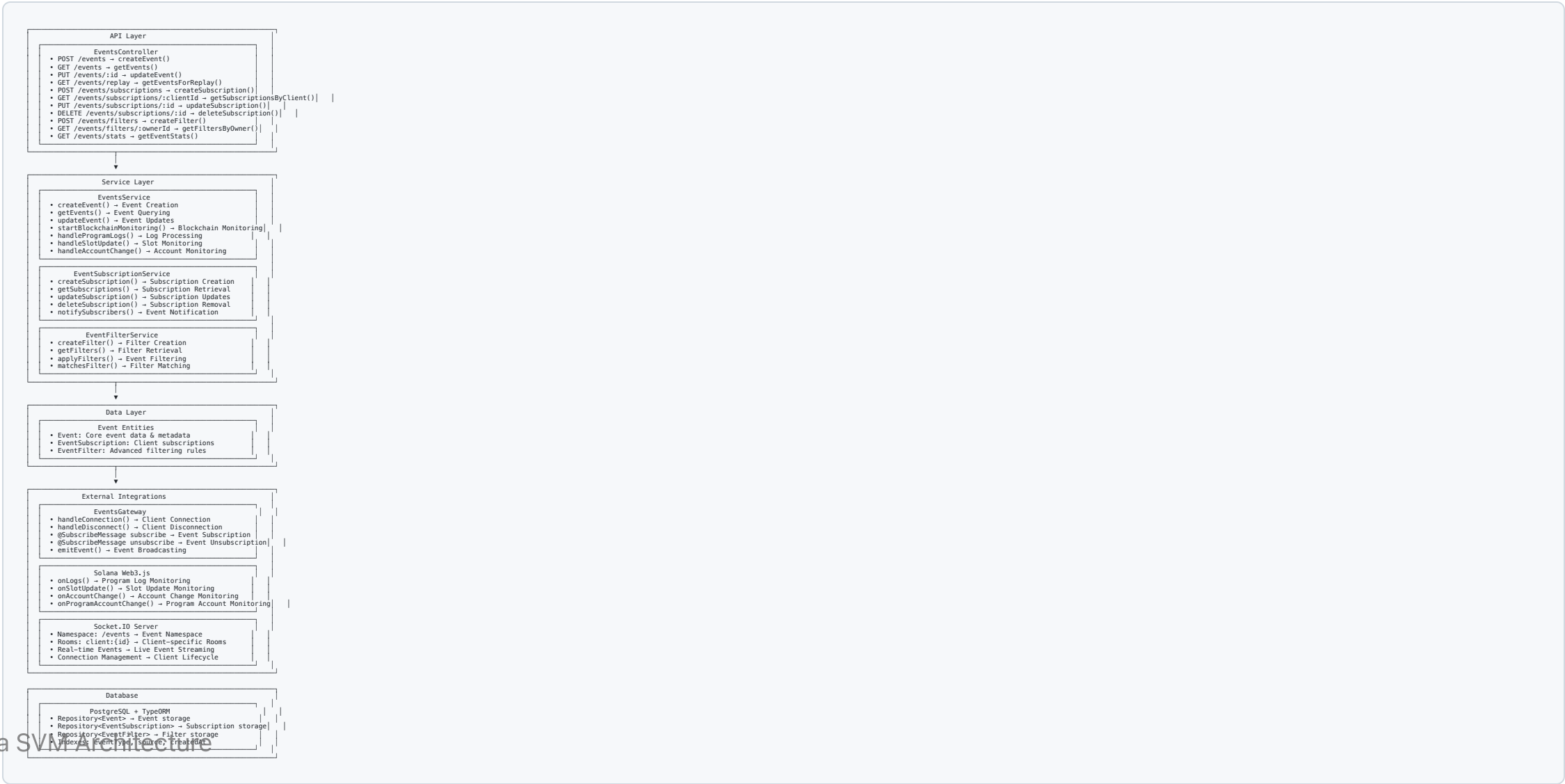
## Event System Overview

- **Real-Time Monitoring:** Live blockchain event streaming
- **WebSocket Integration:** Persistent client connections
- **Subscription Management:** Flexible event filtering and delivery
- **Comprehensive Logging:** Complete audit trail of all operations

## Key Capabilities

- **Blockchain Monitoring:** Program logs, account changes, slot updates
- **Event Filtering:** Advanced filtering and subscription rules
- **Real-Time Delivery:** WebSocket and webhook notifications
- **Historical Replay:** Event replay for state reconstruction

# Architecture Overview



# Event Types & Monitoring

## Supported Event Types

```
enum EventType {  
  TRANSACTION_CONFIRMED = 'TRANSACTION_CONFIRMED',  
  ACCOUNT_CHANGED = 'ACCOUNT_CHANGED',  
  PROGRAM_LOG = 'PROGRAM_LOG',  
  CPI_INVOCATION = 'CPI_INVOCATION',  
  BLOCK_PRODUCED = 'BLOCK_PRODUCED',  
  SLOT_UPDATED = 'SLOT_UPDATED'  
}
```

## Blockchain Monitoring Setup

```
async startBlockchainMonitoring(): Promise<void> {  
  // Program log monitoring  
  this.connection.onLogs('all', (logs) => {  
    this.handleProgramLogs(logs);  
  });  
}
```

# Event Processing Pipeline

## Event Flow Architecture

1. 🔍 Event Detection → Blockchain Monitoring
2. 📝 Event Creation → `createEvent()`
3. 💾 Database Storage → Persistent Storage
4. 📡 WebSocket Emission → Real-time Broadcasting
5. 🎯 Subscription Filtering → Targeted Delivery
6. 🌐 Webhook Delivery → HTTP Callbacks

## Event Entity Structure

```
@Entity()  
export class Event {  
  @PrimaryGeneratedColumn('uuid')  
  id: string;  
  
  @Column({  
    type: 'enum',  
    enum: EventType  
  })  
  eventType: EventType;  
  
  @Column()  
  source: string; // Account/Program ID
```

# Subscription Management

## WebSocket Gateway Implementation

```
@WebSocketGateway({
  namespace: '/events',
  cors: true
})
export class EventsGateway {
  @WebSocketServer()
  server: Server;

  private clients = new Map<string, Socket>();

  handleConnection(client: Socket) {
    this.clients.set(client.id, client);
    client.join(`client:${client.id}`);
  }

  handleDisconnect(client: Socket) {
    this.clients.delete(client.id);
  }

  @SubscribeMessage('subscribe')
  handleSubscribe(client: Socket, data: SubscribeDto) {
    // Create subscription
    const subscription = await this.subscriptionService.createSubscription({
      clientId: client.id,
      eventTypes: data.eventTypes,
      filters: data.filters
    });

    // Join subscription room
    client.join(`subscription:${subscription.id}`);
  }

  emitEvent(event: Event) {
    // Broadcast to all subscribers
  }
}
```

# Advanced Event Filtering

## Filter System Architecture

```
interface EventFilter {
  id: string;
  name: string;
  eventType: EventType;
  conditions: FilterCondition[];
  action: 'include' | 'exclude' | 'transform';
  isActive: boolean;
}

interface FilterCondition {
  field: string;           // Event field to check
  operator: 'eq' | 'ne' | 'gt' | 'lt' | 'contains' | 'regex';
  value: any;              // Comparison value
}
```



# Real-Time Features

## WebSocket Real-Time Streaming

- **Persistent Connections:** Long-lived client connections
- **Room-Based Messaging:** Targeted event broadcasting
- **Connection Recovery:** Automatic reconnection handling
- **Load Balancing:** Scalable WebSocket server architecture
- **Message Buffering:** Event queuing during network issues

## Real-Time Monitoring Capabilities

```
// Account monitoring subscription
@Post('/monitor/account/:accountId')
async subscribeToAccount(
  @Param('accountId') accountId: string,
  @Body() subscription: AccountSubscriptionDto
): Promise<SubscriptionResult> {
  // Start monitoring account changes
```

# Event Replay & Historical Analysis

## Event Replay Implementation

```
async getEventsForReplay(replayDto: EventReplayDto): Promise<Event[]> {
  const query = this.eventRepository.createQueryBuilder('event');

  // Apply time range filter
  if (replayDto.startTime) {
    query.andWhere('event.createdAt >= :startTime', {
      startTime: replayDto.startTime
    });
  }

  if (replayDto.endTime) {
    query.andWhere('event.createdAt <= :endTime', {
      endTime: replayDto.endTime
    });
  }

  // Apply event type filter
  if (replayDto.eventTypes?.length) {
    query.andWhere('event.eventType IN (:...eventTypes)', {
      eventTypes: replayDto.eventTypes
    });
  }

  // Apply source filter
  if (replayDto.sources?.length) {
    query.andWhere('event.source IN (:...sources)', {
      sources: replayDto.sources
    });
  }
}
```

# API Endpoints

## Event Management

- `POST /events` - Create custom event
- `GET /events` - Query events with filtering
- `PUT /events/:id` - Update event status
- `GET /events/replay` - Get events for replay

## Subscription Management

- `POST /events/subscriptions` - Create event subscription
- `GET /events/subscriptions/:clientId` - Get client subscriptions
- `PUT /events/subscriptions/:id` - Update subscription
- `DELETE /events/subscriptions/:id` - Remove subscription

# Key Takeaways

## Event System Benefits

- **Real-Time Monitoring:** Live blockchain event streaming
- **Flexible Subscriptions:** WebSocket and webhook delivery options
- **Advanced Filtering:** Granular event selection and processing
- **Historical Analysis:** Complete event replay and statistics

## SVM Event Advantages

- **High-Frequency Events:** Handle thousands of events per second
- **Parallel Processing:** Multiple event streams processed simultaneously
- **State Synchronization:** Real-time state updates across distributed systems
- **Audit Trail:** Immutable event history for compliance and debugging

# **Module 12: Security and Best Practices**

## **Comprehensive Security Framework**

# Security Architecture Overview

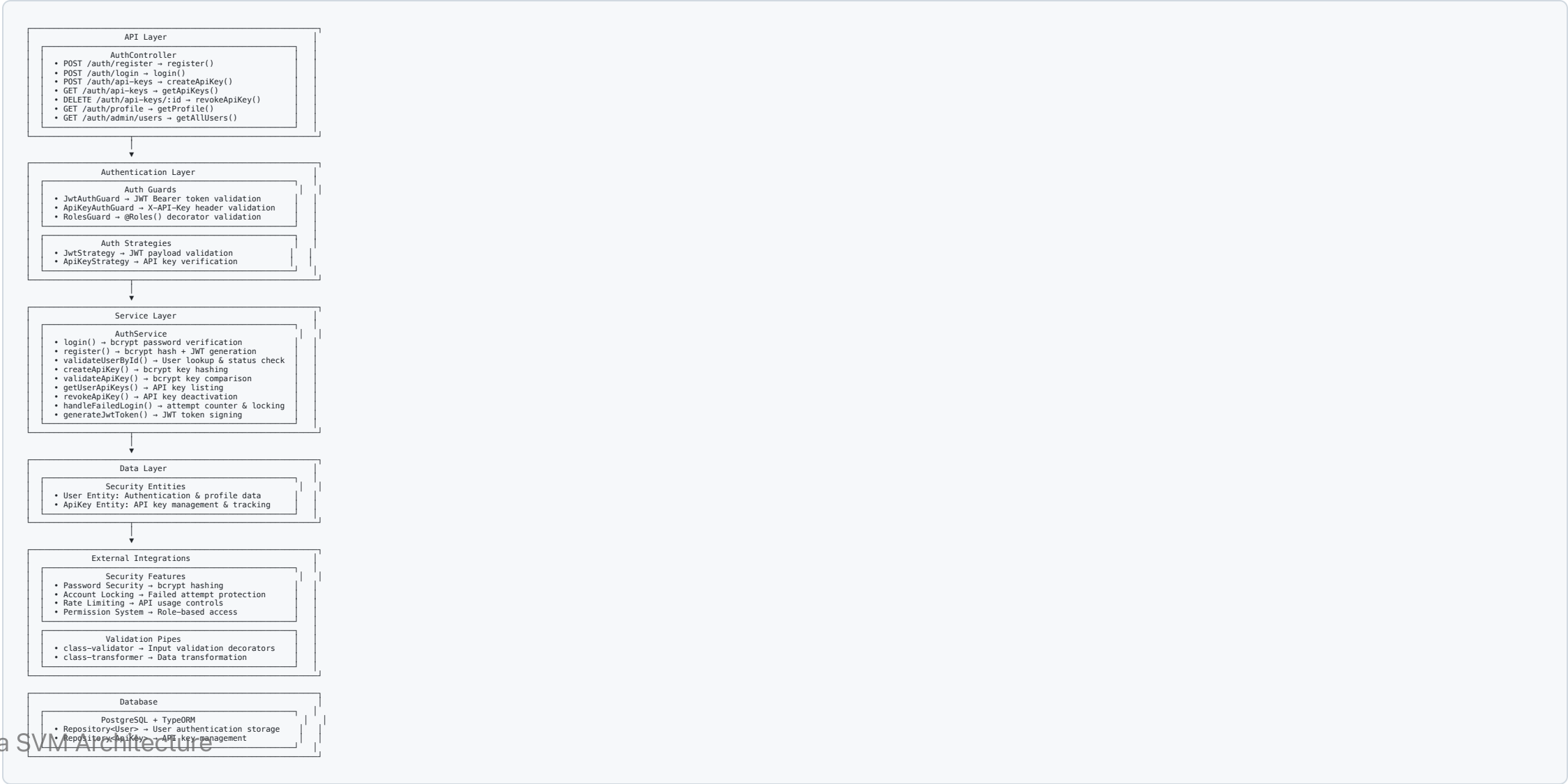
## Multi-Layer Security

- **Authentication:** JWT tokens and API key validation
- **Authorization:** Role-based access control and permissions
- **Input Validation:** Comprehensive data sanitization
- **Rate Limiting:** API usage controls and abuse prevention
- **Audit Logging:** Complete security event tracking

## Security Principles

- **Defense in Depth:** Multiple security layers
- **Least Privilege:** Minimum required permissions
- **Fail-Safe Defaults:** Secure default configurations
- **Complete Auditing:** All security events logged

# System Architecture



# Authentication Guards

## JWT Authentication Guard

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean {
    // Allow access to auth endpoints without token
    const request = context.switchToHttp().getRequest();
    const isAuthEndpoint = request.url.includes('/auth/');

    if (isAuthEndpoint) {
      return true;
    }

    // Validate JWT token for protected endpoints
    return super.canActivate(context);
  }
}
```



# Authentication Strategies

## JWT Strategy Implementation

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: AuthService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET,
    });
  }

  async validate(payload: JwtPayload): Promise<User> {
    const user = await this.authService.validateUserById(payload.sub);

    if (!user) {
      throw new UnauthorizedException('User not found');
    }

    if (user.status !== 'active') {
      throw new UnauthorizedException('User account is not active');
    }

    return user;
  }
}
```

# Security Entities

## User Entity

```
@Entity()
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  @Index()
  email: string;

  @Column()
  passwordHash: string;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;

  @Column({ default: 0 })
  loginAttempts: number;

  @Column({ nullable: true })
  lockedUntil?: Date;

  @Column({
    type: 'enum',
    enum: UserStatus,
    default: UserStatus.ACTIVE
  })
  status: UserStatus;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;

  // Business logic methods
  isLocked(): boolean {
    return this.lockedUntil && this.lockedUntil > new Date();
  }
}
```

# Password Security

## Password Hashing & Verification

```
@Injectable()
export class AuthService {
  private readonly SALT_ROUNDS = 12;

  async hashPassword(password: string): Promise<string> {
    return bcrypt.hash(password, this.SALT_ROUNDS);
  }

  async verifyPassword(password: string, hash: string): Promise<boolean> {
    return bcrypt.compare(password, hash);
  }

  async register(registerDto: RegisterDto): Promise<AuthResponse> {
    // Validate password strength
    this.validatePasswordStrength(registerDto.password);

    // Check if user exists
    const existingUser = await this.userRepository.findOne({
      where: { email: registerDto.email }
    });

    if (existingUser) {
      throw new ConflictException('User already exists');
    }

    // Hash password
    const passwordHash = await this.hashPassword(registerDto.password);

    // Create user
    const user = this.userRepository.create({
      email: registerDto.email,
      passwordHash,
      role: UserRole.USER
    });

    await this.userRepository.save(user);

    // Generate JWT token
    const token = this.generateJwtToken(user);
  }
}
```

# Account Protection

## Failed Login Handling

```

async handleFailedLogin(email: string): Promise<void> {
  const user = await this.userRepository.findOne({ where: { email } });

  if (!user) {
    // Don't reveal if user exists
    return;
  }

  user.loginAttempts += 1;

  // Lock account after 5 failed attempts
  if (user.loginAttempts >= 5) {
    user.lockedUntil = new Date(Date.now() + 15 * 60 * 1000); // 15 minutes
    user.loginAttempts = 0; // Reset counter
  }

  await this.userRepository.save(user);
}

async login(loginDto: LoginDto): Promise<AuthResponse> {
  const user = await this.userRepository.findOne({
    where: { email: loginDto.email }
  });

  if (!user) {
    throw new UnauthorizedException('Invalid credentials');
  }

  // Check if account is locked
  if (user.isLocked()) {
    throw new UnauthorizedException('Account is temporarily locked');
  }

  // Verify password
  const isPasswordValid = await this.verifyPassword(loginDto.password, user.passwordHash);

  if (!isPasswordValid) {
    await this.handleFailedLogin(loginDto.email);
    throw new UnauthorizedException('Invalid credentials');
  }

  // Reset login attempts on successful login
  user.loginAttempts = 0;
  user.lockedUntil = null;
  await this.userRepository.save(user);

  // Generate token

```

# API Key Management

## API Key Creation & Validation

```

async createApiKey(userId: string, createDto: CreateApiKeyDto): Promise<ApiKeyResponse> {
  // Generate random API key
  const apiKeyValue = crypto.randomBytes(32).toString('hex');

  // Hash the full key for storage
  const keyHash = await bcrypt.hash(apiKeyValue, 12);

  // Store first 8 characters for identification
  const keyPrefix = apiKeyValue.substring(0, 8);

  // Create API key entity
  const apiKey = this.apiKeyRepository.create({
    keyHash,
    keyPrefix,
    user: { id: userId } as User,
    permission: createDto.permission || ApiKeyPermission.READ,
    expiresAt: createDto.expiresAt,
    rateLimit: createDto.rateLimit || 100
  });

  await this.apiKeyRepository.save(apiKey);

  // Return the actual key (only time it's shown)
  return {
    id: apiKey.id,
    key: apiKeyValue, // Full key returned only once
    keyPrefix,
    permission: apiKey.permission,
    expiresAt: apiKey.expiresAt,
    rateLimit: apiKey.rateLimit
  };
}

async validateApiKey(apiKeyValue: string): Promise<ApiKeyValidationResult> {
  // Find API key by prefix (first 8 chars)
  const keyPrefix = apiKeyValue.substring(0, 8);

  const apiKey = await this.apiKeyRepository.findOne({
    where: { keyPrefix },
    relations: ['user']
  });

  if (!apiKey || !apiKey.isValid()) {
    return { isValid: false };
  }

  // Verify full key against hash
  const isValid = await bcrypt.compare(apiKeyValue, apiKey.keyHash);

  if (!isValid) {
    return { isValid: false };
  }

  return {
    isValid: true,
    user: apiKey.user,
    permission: apiKey.permission,
    expiresAt: apiKey.expiresAt,
    rateLimit: apiKey.rateLimit
  };
}

```

# Rate Limiting & Usage Tracking

## API Key Rate Limiting

```
async updateApiKeyUsage(apiKeyId: string): Promise<void> {
  const apiKey = await this.apiKeyRepository.findOne({
    where: { id: apiKeyId }
  });

  if (!apiKey) return;

  // Update usage statistics
  apiKey.usageCount += 1;
  apiKey.lastUsedAt = new Date();

  await this.apiKeyRepository.save(apiKey);
}

async checkRateLimit(apiKey: ApiKey): Promise<boolean> {
  if (!apiKey.lastUsedAt) {
    return true; // No usage yet
  }

  const now = new Date();
  const timeSinceLastUse = now.getTime() - apiKey.lastUsedAt.getTime();
  const requestsPerMs = apiKey.rateLimit / 60000; // per minute to per ms

  // Simple rate limiting: ensure minimum time between requests
  const minIntervalMs = 1000 / requestsPerMs;
```

# Input Validation & Sanitization

## DTO Validation with Class Validator

```
import { IsEmail, IsNotEmpty, MinLength, IsEnum, IsOptional } from 'class-validator';

export class RegisterDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  @MinLength(8)
  password: string;

  @IsOptional()
  @IsEnum(UserRole)
  role?: UserRole;
}

export class LoginDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  password: string;
}

export class CreateApiKeyDto {
  @IsOptional()
  @IsEnum(ApiKeyPermission)
  permission?: ApiKeyPermission;

  @IsOptional()
  expiresAt?: Date;
  @IsOptional()
  @Min(1)
```

# API Endpoints

## Authentication Endpoints

- `POST /auth/register` - User registration with password validation
- `POST /auth/login` - User login with account locking protection
- `GET /auth/profile` - Get current user profile
- `GET /auth/admin/users` - Admin endpoint for user management

## API Key Management

- `POST /auth/api-keys` - Create new API key
- `GET /auth/api-keys` - List user's API keys
- `DELETE /auth/api-keys/:id` - Revoke API key



# Key Takeaways

## Security Framework Benefits

- **Multi-Factor Authentication:** JWT + API key support
- **Comprehensive Protection:** Password security, account locking, rate limiting
- **Role-Based Access:** Hierarchical permission system
- **Complete Auditing:** All security events tracked

## Implementation Best Practices

- **bcrypt Hashing:** Industry-standard password security
- **JWT Tokens:** Stateless authentication with expiration
- **Input Validation:** Prevent injection and malformed data
- **Rate Limiting:** Prevent abuse and DoS attacks
- **Audit Logging:** Complete security event tracking

# **Module 13: Development Tools and Frameworks**

## **Complete Development Environment**

# Development Stack Overview

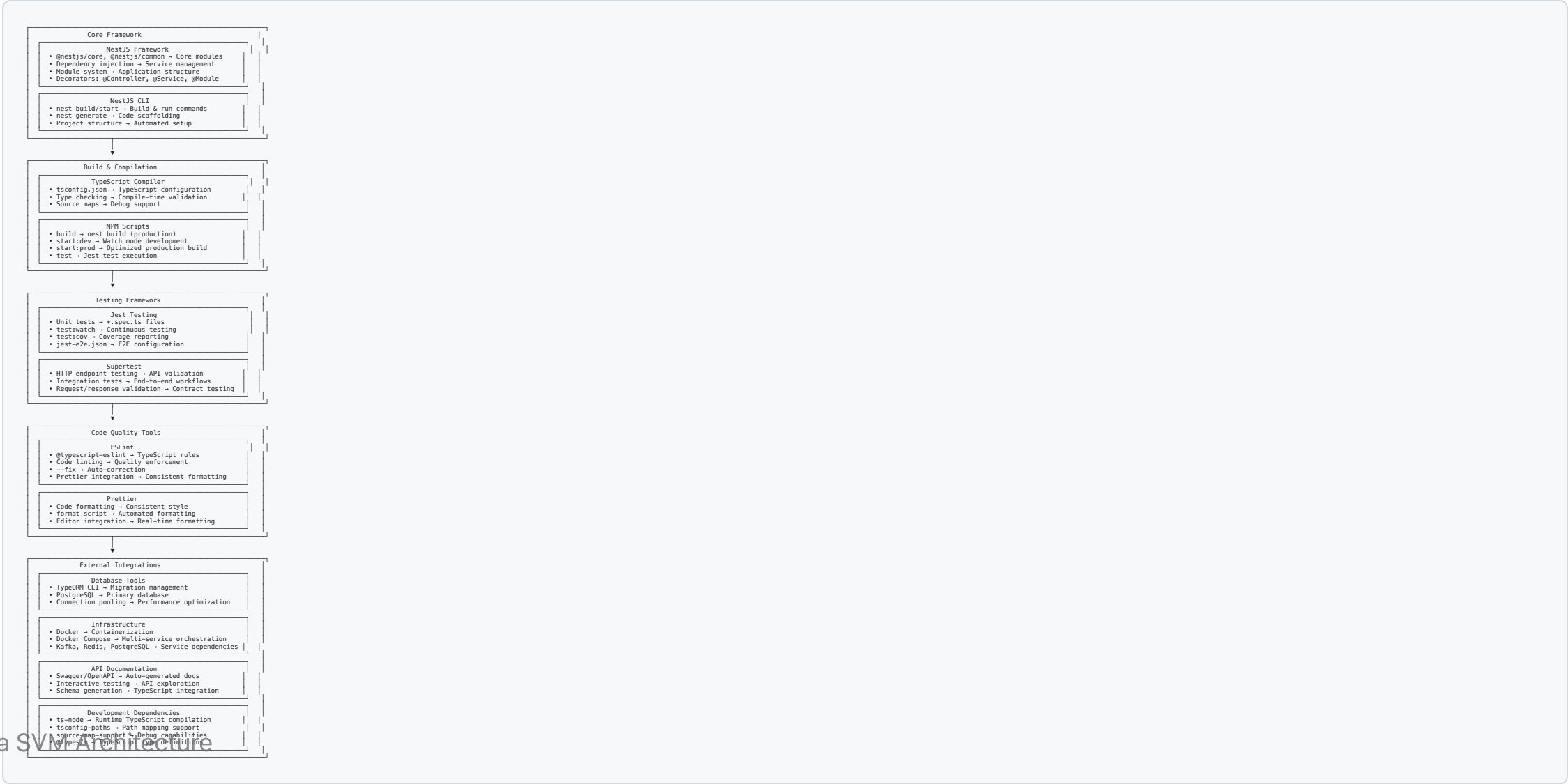
## Core Technologies

- **NestJS:** Node.js framework for scalable server-side applications
- **TypeScript:** Typed JavaScript for better development experience
- **PostgreSQL:** Advanced open-source relational database
- **Redis:** In-memory data structure store for caching
- **Kafka:** Distributed event streaming platform
- **Docker:** Containerization for consistent environments

## Development Principles

- **Type Safety:** Full TypeScript integration
- **Test-Driven Development:** Comprehensive testing framework
- **Code Quality:** Automated linting and formatting

# Architecture Overview



# NestJS Framework

## Core Architecture

```
// Main application module
@Module({
  imports: [
    ConfigModule.forRoot(),
    TypeOrmModule.forRoot(databaseConfig),
    AuthModule,
    AccountsModule,
    TransactionsModule
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}

// Controller with dependency injection
@Controller('accounts')
export class AccountsController {
  constructor(private readonly accountsService: AccountsService) {}

  @Get()
  async findAll(): Promise<Account[]> {
    return this.accountsService.findAll();
  }

  @Post()
  async create(@Body() createAccountDto: CreateAccountDto): Promise<Account> {
    return this.accountsService.create(createAccountDto);
  }
}

// Service with business logic
@Injectable()
export class AccountsService {
  constructor(
    @InjectRepository(Account)
    private accountsRepository: Repository<Account>,
  ) {}

  async findAll(): Promise<Account[]> {
    return this.accountsRepository.find();
  }

  async create(createAccountDto: CreateAccountDto): Promise<Account> {

```

# TypeScript Configuration

## tsconfig.json

```
{
  "compilerOptions": {
    "module": "commonjs",
    "declaration": true,
    "removeComments": true,
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true,
    "allowSyntheticDefaultImports": true,
    "target": "ES2020",
    "sourceMap": true,
    "outDir": "./dist",
    "baseUrl": "./",
    "incremental": true,
    "skipLibCheck": true,
    "strictNullChecks": false,
    "noImplicitAny": false,
    "strictBindCallApply": false,
    "forceConsistentCasingInFileNames": false,
    "noFallthroughCasesInSwitch": false,
    "paths": {
      "@/*": ["src/*"],
      "@common/*": ["src/common/*"],
      "@modules/*": ["src/modules/*"]
    },
    "include": ["src/**/*.ts"],
  },
}
```

# Testing Framework

## Jest Configuration

```
// jest.config.js
module.exports = {
  moduleFileExtensions: ['js', 'json', 'ts'],
  rootDir: 'src',
  testRegex: '.*\\.spec\\.ts$',
  transform: {
    '^.+\\.?(t|j)s$': 'ts-jest',
  },
  collectCoverageFrom: [
    '**/*.?(t|j)s',
    '!**/*.module.ts',
    '!**/main.ts',
  ],
  coverageDirectory: '../coverage',
  testEnvironment: 'node',
  moduleNameMapping: {
    '@/(.*)$': '<rootDir>/$1',
  }
}
```

# Code Quality Tools

## ESLint Configuration

```
// .eslintrc.js
module.exports = {
  parser: '@typescript-eslint/parser',
  parserOptions: {
    project: 'tsconfig.json',
    tsconfigRootDir: __dirname,
    sourceType: 'module',
  },
  plugins: ['@typescript-eslint/eslint-plugin'],
  extends: [
    'eslint:recommended',
    '@typescript-eslint/recommended',
    'prettier',
  ],
  root: true,
  env: {
    node: true,
    jest: true,
  },
  ignorePatterns: ['.eslintrc.js'],
  rules: {
    '@typescript-eslint/interface-name-prefix': 'off',
    '@typescript-eslint/explicit-function-return-type': 'off',
    '@typescript-eslint/explicit-module-boundary-types': 'off',
    '@typescript-eslint/no-explicit-any': 'off',
  }
}
```



# Database Tools

## TypeORM Configuration

```
// data-source.ts
export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT) || 5432,
  username: process.env.DB_USERNAME || 'postgres',
  password: process.env.DB_PASSWORD || 'password',
  database: process.env.DB_NAME || 'solana_svm',
  synchronize: false, // Use migrations in production
  logging: process.env.NODE_ENV === 'development',
  entities: ['dist/**/*.entity{.ts,.js}'],
  migrations: ['dist/database/migrations/*{.ts,.js}'],
  subscribers: ['dist/database/subscribers/*{.ts,.js}'],
});
```

# Docker & Infrastructure

## Multi-Stage Dockerfile

```
# Development stage
FROM node:18-alpine AS development
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine AS production
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY --from=development /app/dist ./dist
COPY --from=development /app/migrations ./migrations
USER node
EXPOSE 3000
```

# API Documentation

## Swagger Integration

```
// main.ts
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  const config = new DocumentBuilder()
    .setTitle('Solana SVM Study API')
    .setDescription('API for Solana SVM blockchain operations')
    .setVersion('1.0')
    .addTag('accounts', 'Account management endpoints')
    .addTag('transactions', 'Transaction operations')
    .addTag('signing', 'Cryptographic signing operations')
    .addBearerAuth()
    .addApiKey({ type: 'apiKey', name: 'X-API-Key', in: 'header' }, 'api-key')
    .build();

  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(3000);
}
```

# Development Workflow

## Development Scripts

```
{
  "scripts": {
    "build": "nest build",
    "format": "prettier --write \"src/**/*.ts\" \"test/**/*.ts\"",
    "start": "nest start",
    "start:dev": "nest start --watch",
    "start:debug": "nest start --debug --watch",
    "start:prod": "node dist/main",
    "lint": "eslint \"{src,apps,libs,test}/**/*.ts\" --fix",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:cov": "jest --coverage",
    "test:debug": "node --inspect-brk -r tsconfig-paths/register -r ts-node/register node_modules/.bin/jest --runInBand",
    "test:e2e": "jest --config ./test/jest-e2e.json"
  }
}
```

## Development Environment Setup

```
# Install dependencies
npm install
```

```
# Start development database
```

# Key Takeaways

## Development Environment Benefits

- **Full-Stack TypeScript:** Type safety from database to API
- **Comprehensive Testing:** Unit and integration test coverage
- **Code Quality Automation:** Linting and formatting enforcement
- **Containerized Development:** Consistent environments across teams
- **API Documentation:** Auto-generated, interactive documentation

## Tool Integration Benefits

- **NestJS Framework:** Scalable, maintainable application structure
- **TypeORM:** Type-safe database operations with migrations
- **Jest + Supertest:** Complete testing framework for APIs
- **Docker Compose:** Multi-service development environment

# **Module 14: Network Architecture**

## **Scalable Distributed System Design**

# System Architecture Overview

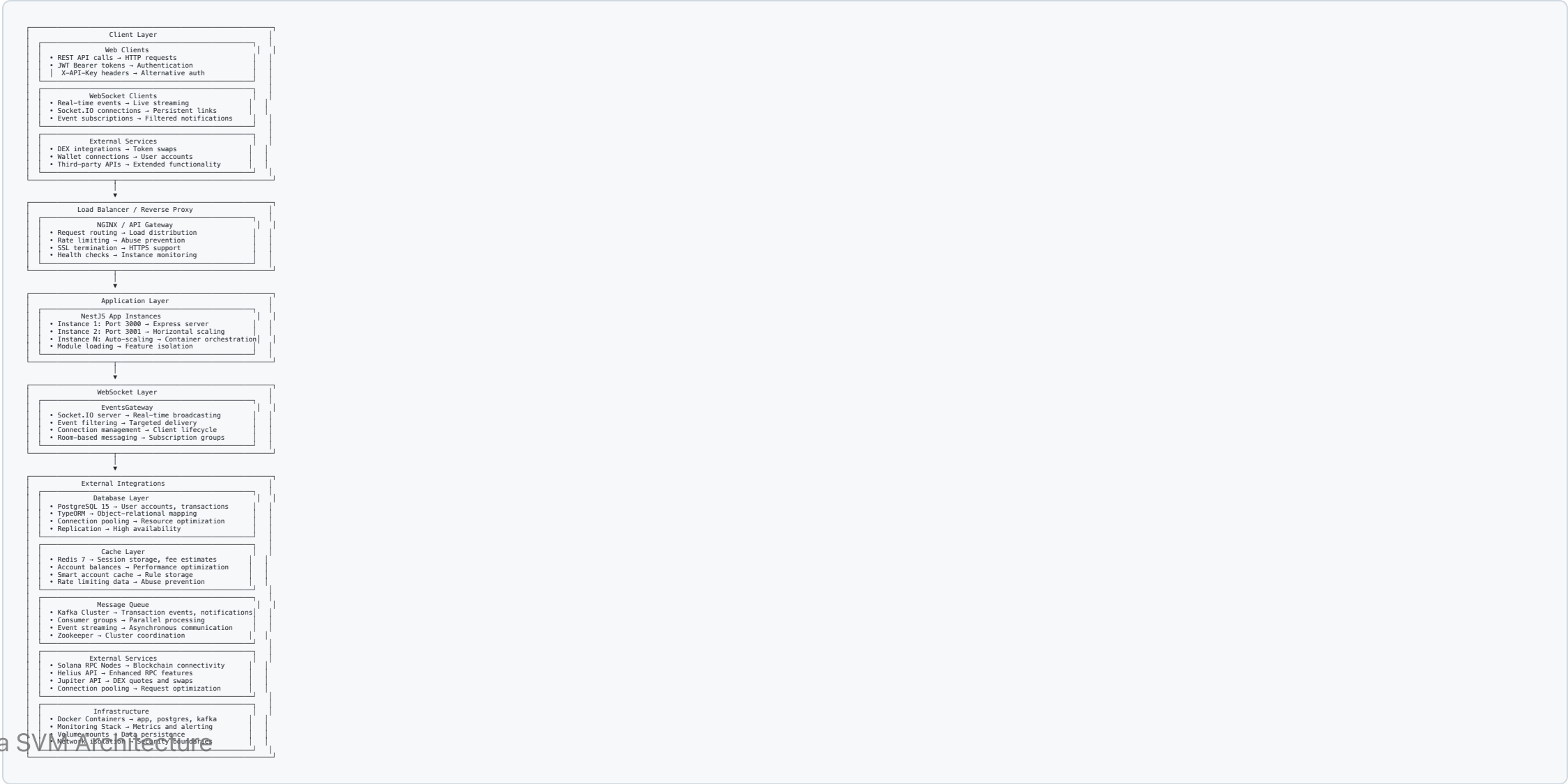
## Architecture Principles

- **Microservices Design:** Modular, independently deployable services
- **Horizontal Scaling:** Multiple application instances behind load balancer
- **Event-Driven Communication:** Asynchronous messaging with Kafka
- **Multi-Layer Caching:** Redis for performance optimization
- **External API Integration:** Solana RPC and DEX service connections

## Scalability Features

- **Load Balancing:** Request distribution across multiple instances
- **Database Connection Pooling:** Efficient database resource management
- **WebSocket Broadcasting:** Real-time event distribution
- **Message Queuing:** Asynchronous processing and decoupling

# System Architecture





# Load Balancing & Scaling

## NGINX Configuration

```
# nginx.conf
upstream backend {
    least_conn;
    server app:3000;
    server app:3001;
    server app:3002;
}

server {
    listen 80;
    server_name api.solana-svm-study.com;

    # Rate limiting
    limit_req zone=api burst=10 nodelay;

    # SSL termination
    listen 443 ssl http2;
    ssl_certificate /etc/ssl/certs/api.crt;
    ssl_certificate_key /etc/ssl/private/api.key;

    # API routing
    location /api/ {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket routing
    location /socket.io/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Health checks
    location /health {
        access_log off;
    }
}
```

# Database Architecture

## PostgreSQL Configuration

```
// Connection pooling with TypeORM
export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT),
  username: process.env.DB_USERNAME,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,

  // Connection pool settings
  extra: {
    max: 20,           // Maximum connections
    min: 5,            // Minimum connections
    idleTimeoutMillis: 30000,
    acquireTimeoutMillis: 60000,
  },

  // Replication for read scaling
  replication: {
    master: {
      host: process.env.DB_MASTER_HOST,
      port: parseInt(process.env.DB_MASTER_PORT),
      username: process.env.DB_MASTER_USER,
      password: process.env.DB_MASTER_PASSWORD,
    },
    slaves: [{
      host: process.env.DB_SLAVE_HOST,
      port: parseInt(process.env.DB_SLAVE_PORT),
      username: process.env.DB_SLAVE_USER,
      password: process.env.DB_SLAVE_PASSWORD,
    }],
  },
  synchronize: false,
  logging: process.env.NODE_ENV === 'development',
});
```

# Caching Strategy

## Redis Implementation

```
// Redis service for multi-purpose caching
@Injectable()
export class RedisService {
  constructor(private readonly redis: Redis) {}

  // Session storage
  async setSession(sessionId: string, data: any, ttl: number = 3600): Promise<void> {
    await this.redis.setex(`session:${sessionId}`, ttl, JSON.stringify(data));
  }

  async getSession(sessionId: string): Promise<any> {
    const data = await this.redis.get(`session:${sessionId}`);
    return data ? JSON.parse(data) : null;
  }

  // Fee estimation cache
  async cacheFeeEstimate(txSignature: string, estimate: FeeEstimate): Promise<void> {
    await this.redis.setex(`fee:${txSignature}`, 300, JSON.stringify(estimate)); // 5 min TTL
  }

  async getCachedFeeEstimate(txSignature: string): Promise<FeeEstimate | null> {
    const data = await this.redis.get(`fee:${txSignature}`);
    return data ? JSON.parse(data) : null;
  }

  // Account balance cache
  async cacheAccountBalance(address: string, balance: number): Promise<void> {
    await this.redis.setex(`balance:${address}`, 60, balance.toString()); // 1 min TTL
  }

  async getCachedAccountBalance(address: string): Promise<number | null> {
    const balance = await this.redis.get(`balance:${address}`);
    return balance ? parseInt(balance) : null;
  }

  // Rate limiting
  async checkRateLimit(key: string, limit: number, window: number): Promise<boolean> {
    const count = await this.redis.incr(key);
    if (count === 1) {
      await this.redis.expire(key, window);
    }
  }
}
```

# Message Queue Architecture

## Kafka Configuration

```
// Kafka producer configuration
export const kafkaConfig = {
  clientId: 'solana-svm-study',
  brokers: ['kafka:9092'],
  retry: {
    initialRetryTime: 100,
    retries: 8,
  },
};

// Producer service
@Injectable()
export class KafkaProducerService {
  private producer: Producer;

  async onModuleInit() {
    this.producer = this.kafka.producer();
    await this.producer.connect();
  }

  async publishTransactionEvent(event: TransactionEvent): Promise<void> {
    await this.producer.send({
      topic: 'transaction-events',
      messages: [{
        key: event.signature,
        value: JSON.stringify(event),
      }],
    });
  }

  async publishAccountEvent(event: AccountEvent): Promise<void> {
    await this.producer.send({
      topic: 'account-events',
      messages: [{
        key: event.address,
        value: JSON.stringify(event),
      }],
    });
  }
}
```

# WebSocket Architecture

## Real-Time Event Broadcasting

```

@WebSocketGateway({
  namespace: '/events',
  cors: { origin: '*' },
  transports: ['websocket', 'polling'],
})
export class EventsGateway implements OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer()
  server: Server;

  private clients = new Map<string, Socket>();

  handleConnection(client: Socket): void {
    this.clients.set(client.id, client);
    client.join('client:${client.id}');
    console.log(`Client connected: ${client.id}`);
  }

  handleDisconnect(client: Socket): void {
    this.clients.delete(client.id);
    console.log(`Client disconnected: ${client.id}`);
  }

  @SubscribeMessage('subscribe')
  handleSubscribe(client: Socket, payload: SubscribePayload): void {
    const { eventTypes, filters } = payload;

    // Create subscription
    const subscription = this.subscriptionService.createSubscription({
      clientId: client.id,
      eventTypes,
      filters,
    });

    // Join subscription room
    client.join('subscription:${subscription.id}');

    // Send confirmation
    client.emit('subscribed', { subscriptionId: subscription.id });
  }

  @SubscribeMessage('unsubscribe')
  handleUnsubscribe(client: Socket, payload: { subscriptionId: string }): void {
    client.leave('subscription:${payload.subscriptionId}');
    this.subscriptionService.deleteSubscription(payload.subscriptionId);
  }

  async broadcastEvent(event: Event): Promise<void> {
    // Get active subscriptions for this event type
    const subscriptions = await this.subscriptionService.getSubscriptionsByEventType(
      event.eventType
    );
    // Broadcast to matching subscriptions
    for (const subscription of subscriptions) {
      if (this.matchesFilters(event, subscription.filters)) {

```

# External Service Integration

## Solana RPC Connection Pooling

```

@Injectables()
export class SolanaRpcService {
  private connections: Connection[] = [];

  constructor() {
    // Initialize connection pool
    const rpcUrls = [
      'https://api.mainnet-beta.solana.com',
      'https://solana-api.projectserum.com',
      'https://rpc.ankr.com/solana',
    ];

    for (const url of rpcUrls) {
      this.connections.push(new Connection(url, 'confirmed'));
    }
  }

  // Round-robin connection selection
  private getConnection(): Connection {
    const index = Math.floor(Math.random() * this.connections.length);
    return this.connections[index];
  }

  async getAccountInfo(address: PublicKey): Promise<AccountInfo<Buffer> | null> {
    const connection = this.getConnection();
    return await connection.getAccountInfo(address);
  }

  async getBalance(address: PublicKey): Promise<number> {
    const connection = this.getConnection();
    return await connection.getBalance(address);
  }

  async getRecentBlockhash(): Promise<{ blockhash: string; lastValidBlockHeight: number }> {
    const connection = this.getConnection();

```

# Monitoring & Observability

## Application Metrics

```
// Prometheus metrics
import { register, collectDefaultMetrics, Gauge, Counter, Histogram } from 'prom-client';

// Enable default metrics
collectDefaultMetrics();

// Custom metrics
export const httpRequestDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
});

export const activeConnections = new Gauge({
  name: 'websocket_active_connections',
  help: 'Number of active WebSocket connections',
});

export const databaseConnections = new Gauge({
  name: 'database_active_connections',
  help: 'Number of active database connections',
});

export const kafkaMessagesProcessed = new Counter({
  name: 'kafka_messages_processed_total',
  help: 'Total number of Kafka messages processed',
});
```

# Docker & Container Orchestration

## Production Docker Compose

```

version: '3.8'
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000-3002:3000" # Multiple ports for scaling
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
      - REDIS_HOST=redis
      - KAFKA_BROKER=kafka:9092
    depends_on:
      - postgres
      - redis
      - kafka
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure

  postgres:
    image: postgres:15-alpine
    environment:
      - POSTGRES_DB=solana_svm
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 512M

  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    deploy:
      resources:
        limits:
          memory: 512M

  kafka:
    image: confluentinc/cp-kafka:7.3.0
    environment:
      - KAFKA_BROKER_ID=1
      - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
      - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
      - KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1
    depends_on:
      - zookeeper
    volumes:
      - kafka_data:/var/lib/kafka/data

  zookeeper:
    image: confluentinc/cp-zookeeper:7.3.0
    environment:
      - ZOOKEEPER_CLIENT_PORT=2181

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/ssl/certs
    depends_on:

```



# Key Takeaways

## Architecture Benefits

- **Scalability:** Horizontal scaling with load balancing
- **Reliability:** Multiple instances prevent single points of failure
- **Performance:** Caching, connection pooling, and async processing
- **Maintainability:** Modular design with clear separation of concerns
- **Observability:** Comprehensive monitoring and health checks

## Production-Ready Features

- **Container Orchestration:** Docker-based deployment and scaling
- **Database Optimization:** Connection pooling and read replicas
- **Message Queuing:** Asynchronous processing with Kafka
- **Real-Time Communication:** WebSocket broadcasting for live updates

# **Module 15: Advanced Features**

## **Cutting-Edge Blockchain Capabilities**

# Advanced Features Overview

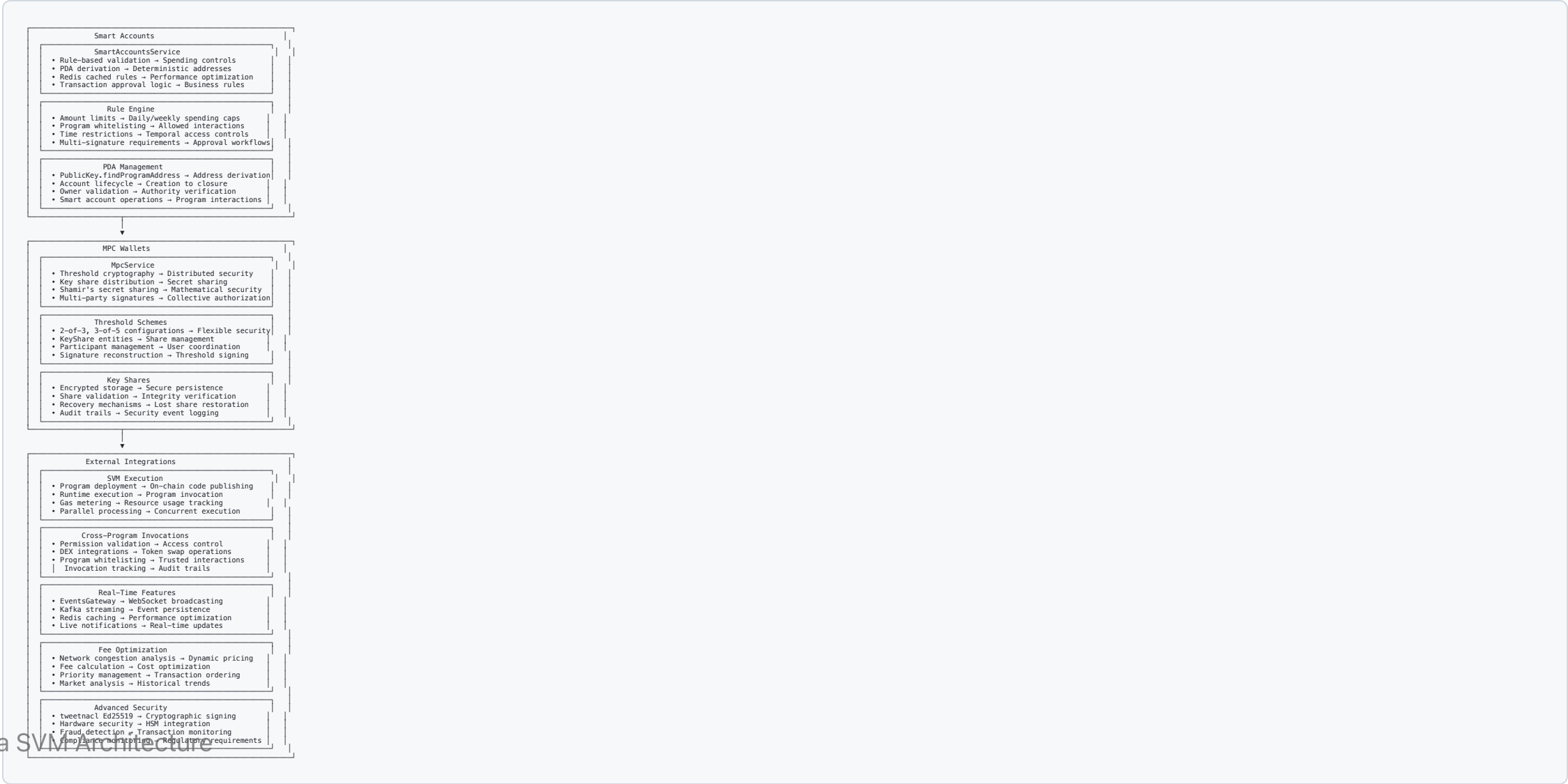
## Next-Generation Capabilities

- **Smart Accounts:** Programmatic account control with validation rules
- **MPC Wallets:** Multi-party computation for enhanced security
- **SVM Execution:** Advanced program runtime with gas metering
- **Cross-Program Invocations:** Secure inter-program communication
- **Real-Time Features:** Live event streaming and notifications
- **Fee Optimization:** Dynamic pricing and cost optimization
- **Advanced Security:** Hardware security and fraud prevention

## Innovation Focus

- **DeFi Integration:** Decentralized exchange operations
- **Institutional Features:** Enterprise-grade security and compliance

# System Architecture



# Smart Accounts Implementation

## Rule-Based Validation Engine

```

@Injectable()
export class SmartAccountsService {
  constructor(
    private redis: Redis,
    private solanaConnection: Connection,
  ) {}

  async validateTransaction(
    smartAccountAddress: string,
    transaction: Transaction
  ): Promise<ValidationResult> {

    // Load account rules from cache
    const rules = await this.getAccountRules(smartAccountAddress);
    if (!rules) {
      return { valid: false, reason: 'Smart account not found' };
    }

    // Check account status
    if (rules.status !== 'ACTIVE') {
      return { valid: false, reason: 'Account is not active' };
    }

    // Validate spending limits
    const dailySpent = await this.getDailySpending(smartAccountAddress);
    const transactionAmount = this.calculateTransactionAmount(transaction);

    if (dailySpent + transactionAmount > rules.maxDailySpend) {
      return { valid: false, reason: 'Daily spending limit exceeded' };
    }

    // Check program whitelist
    const allowedPrograms = new Set(rules.allowedPrograms);
    for (const instruction of transaction.instructions) {
      if (!allowedPrograms.has(instruction.programId.toString())) {
        return { valid: false, reason: 'Program not whitelisted' };
      }
    }

    // Check time restrictions
    if (rules.timeRestrictions) {
      const now = new Date();
      if (!this.isWithinTimeWindow(now, rules.timeRestrictions)) {
        return { valid: false, reason: 'Transaction outside allowed time window' };
      }
    }

    // Multi-signature check
    if (rules.requiredSigners > 1) {
      const signatureCount = transaction.signatures.length;
      if (signatureCount < rules.requiredSigners) {
        return { valid: false, reason: 'Insufficient signatures' };
      }
    }

    return { valid: true };
  }

  private async getAccountRules(address: string): Promise<SmartAccountRules | null> {
    const cached = await this.redis.get(`smart-account:${address}:rules`);
    if (cached) {
      return JSON.parse(cached);
    }

    // Load from database if not cached
    const account = await this.smartAccountRepository.findOne({ where: { address } });
    if (account) {
      await this.redis.setex(`smart-account:${address}:rules`, 3600, JSON.stringify(account.rules));
    }
  }
}

```

# MPC Wallet Implementation

## Threshold Cryptography Service

```
@Injectable()
export class MpcService {
  constructor(
    private encryption: EncryptionService,
    private keySharesRepository: Repository<KeyShare>,
  ) {}

  async createMpcWallet(
    participants: string[],
    threshold: number,
    totalShares: number
  ): Promise<MpcWallet> {
    // Generate master key
    const masterKey = KeyPair.generate();

    // Split key using Shamir's secret sharing
    const shares = this.splitKey(masterKey.secretKey, totalShares, threshold);

    // Encrypt and distribute shares
    const keyShares: KeyShare[] = [];
    for (let i = 0; i < participants.length; i++) {
      const encryptedShare = await this.encryption.encrypt(
        shares[i],
        participants[i] // Encrypt with participant's public key
      );
      keyShares.push(this.keySharesRepository.create({
        walletId: masterKey.publicKey.toString(),
        participantId: participants[i],
        shareIndex: i,
        encryptedShare,
        status: 'ACTIVE'
      }));
    }

    await this.keySharesRepository.save(keyShares);

    return {
      address: masterKey.publicKey.toString(),
      threshold,
      totalShares,
      participants,
      status: 'CREATED'
    };
  }

  async signWithMpc(
    walletAddress: string,
    message: Uint8Array,
    participantSignatures: ParticipantSignature[]
  ): Promise<SignatureResult> {
    // Verify threshold met
    if (participantSignatures.length < this.threshold) {
      throw new BadRequestException('Insufficient signatures for threshold');
    }

    // Decrypt shares from signatures
    const decryptedShares: Uint8Array[] = [];
    for (const sig of participantSignatures) {
      const share = await this.keySharesRepository.findOne({
        where: {
          walletId: walletAddress,
          participantId: sig.participantId
        }
      });
      const decryptedShare = await this.encryption.decrypt(
        share.encryptedShare,
        sig.signature // Use signature as decryption key
      );
      decryptedShares.push(decryptedShare);
    }

    // Reconstruct private key
    const reconstructedKey = this.reconstructKey(decryptedShares);

    // Sign message
    const signature = nacl.sign.detached(message, reconstructedKey);

    return {
      signature: Buffer.from(signature).toString('base64'),
      publicKey: walletAddress
    };
  }

  private splitKey(secretKey: Uint8Array, n: number, t: number): Uint8Array[] {
    // Implementation of Shamir's secret sharing
    // This is a simplified version - production would use a proper crypto library
    const shares: Uint8Array[] = [];

    for (let i = 0; i < n; i++) {
      // Generate polynomial coefficients
      const coefficients = [secretKey];
      // ... (simplified implementation) ...
      // Evaluate polynomial at point i+1
      const share = this.evaluatePolynomial(coefficients, i + 1);
      shares.push(share);
    }
  }
}
```

# SVM Execution Engine

## Gas Metering and Resource Management

```

@Injectable()
export class SvmService {
  constructor(private connection: Connection) {}

  async executeProgram(params: ProgramExecutionParams): Promise<ExecutionResult> {
    const startTime = Date.now();
    let computeUnitsUsed = 0;

    try {
      // Estimate compute units
      const estimatedCU = await this.estimateComputeUnits(params.instruction);

      // Check against limits
      if (estimatedCU > MAX_COMPUTE_UNITS) {
        throw new BadRequestException('Instruction exceeds compute unit limit');
      }

      // Set compute unit limit
      const setCULimitTx = ComputeBudgetProgram.setComputeUnitLimit({
        units: estimatedCU
      });

      // Build transaction with compute budget
      const transaction = new Transaction()
        .add(setCULimitTx)
        .add(params.instruction);

      // Add accounts
      transaction.recentBlockhash = (await this.connection.getRecentBlockhash()).blockhash;
      transaction.feePayer = params.feePayer;

      // Sign and send
      const signature = await this.connection.sendAndConfirmTransaction(transaction, [params.signer]);

      // Get actual compute units used (would need program logs parsing)
      computeUnitsUsed = await this.parseComputeUnitsFromLogs(signature);

      return {
        signature,
        success: true,
        computeUnitsUsed,
        executionTime: Date.now() - startTime
      };
    } catch (error) {
      return {
        signature: null,
        success: false,
        error: error.message,
        computeUnitsUsed,
        executionTime: Date.now() - startTime
      };
    }
  }

  private async estimateComputeUnits(instruction: TransactionInstruction): Promise<number> {
    const programId = instruction.programId.toString();

    // Base overhead
    let cu = 5000;

    // Program-specific estimates
    if (programId === SystemProgram.programId.toString()) {
      cu += this.estimateSystemProgramCU(instruction);
    } else if (programId === TOKEN_PROGRAM_ID.toString()) {
      cu += this.estimateTokenProgramCU(instruction);
    } else {
      // Conservative estimate for unknown programs
      cu += 10000;
    }

    return cu;
  }

  private estimateSystemProgramCU(instruction: TransactionInstruction): number {

```

# DEX Integration and CPI

## Decentralized Exchange Operations

```

@Injectable()
export class DexService {
  constructor(
    private cpiService: CpiService,
    private jupiterApi: JupiterApiService,
  ) {}

  async executeTokenSwap(swapParams: TokenSwapParams): Promise<SwapResult> {
    // Get quote from Jupiter
    const quote = await this.jupiterApi.getQuote({
      inputMint: swapParams.inputMint,
      outputMint: swapParams.outputMint,
      amount: swapParams.amount,
      slippageBps: swapParams.slippage * 100, // Convert to basis points
    });

    // Validate slippage
    if (quote.priceImpactPct > swapParams.maxPriceImpact) {
      throw new BadRequestException('Price impact too high');
    }

    // Create swap instruction via CPI
    const swapInstruction = await this.createJupiterSwapInstruction(quote);

    // Execute via CPI service
    const result = await this.cpiService.executeCpi({
      callerProgramId: this.dexProgramId,
      targetProgramId: quote.programId,
      instruction: swapInstruction,
      accounts: quote.accounts,
      requiresPermission: false, // DEX swaps are typically permissionless
    });

    return {
      signature: result.transactionId,
      inputAmount: swapParams.amount,
      outputAmount: quote.outputAmount,
      priceImpact: quote.priceImpactPct,
      fee: quote.fee
    };
  }

  async getLiquidityPools(): Promise<LiquidityPool[]> {
    // Query Jupiter for available pools
    const pools = await this.jupiterApi.getPools();

    return pools.map(pool => ({
      id: pool.id,
      tokenA: pool.tokenA,
      tokenB: pool.tokenB,
      liquidity: pool.liquidity,
      fee: pool.fee,
      volume24h: pool.volume24h
    }));
  }

  private async createJupiterSwapInstruction(quote: JupiterQuote): Promise<TransactionInstruction> {
    // This would integrate with Jupiter's instruction builder
    // Simplified for demonstration
    return new TransactionInstruction({
      programId: new PublicKey(quote.programId),
      keys: quote.accounts.map(acc => ({
        pubkey: new PublicKey(acc.pubkey),
        isSigner: acc.isSigner,
      })),
    });
  }
}

```



# Real-Time Event Streaming

## WebSocket Event Broadcasting

```

@Injectable()
export class RealTimeEventsService {
  constructor(
    private eventsGateway: EventsGateway,
    private kafkaProducer: KafkaProducerService,
    private redis: Redis,
  ) {}

  async broadcastTransactionEvent(event: TransactionEvent): Promise<void> {
    // Cache event for replay
    await this.redis.setex(
      `event:${event.signature}`,
      86400, // 24 hours
      JSON.stringify(event)
    );

    // Publish to Kafka for persistence
    await this.kafkaProducer.publishTransactionEvent(event);

    // Broadcast via WebSocket
    await this.eventsGateway.broadcastEvent({
      eventType: EventType.TRANSACTION_CONFIRMED,
      source: event.signature,
      data: event,
      slot: event.slot,
      signature: event.signature,
    });
  }

  async subscribeToAccount(accountAddress: string, clientId: string): Promise<void> {
    // Create subscription
    const subscription = await this.subscriptionService.createSubscription({
      clientId,
      eventTypes: [EventType.ACCOUNT_CHANGED],
      filters: {
        accountAddress,
      },
    });

    // Start monitoring account changes
    await this.connection.onAccountChange(
      new PublicKey(accountAddress),
      async (accountInfo) => {
        const event: AccountEvent = {
          address: accountAddress,
          data: accountInfo,
          slot: accountInfo.slot,
          timestamp: new Date(),
        };
        await this.broadcastAccountEvent(event);
      }
    );
  }

  async getEventHistory(
    eventType: EventType,
    startTime: Date,
    endTime: Date
  ): Promise<Event[]> {
    // Query from database with time range
    return await this.eventRepository.find({
      where: {
        eventType,
        Between(startTime, endTime)
      },
    });
  }
}

```

# Fee Optimization Engine

## Dynamic Fee Calculation

```
@Injectable()
export class FeeOptimizationService {
  constructor(
    private connection: Connection,
    private redis: Redis,
  ) {}

  async optimizeFee(
    transaction: Transaction,
    strategy: FeeOptimizationStrategy = 'BALANCED'
  ): Promise<OptimizedFee> {
    // Get network congestion
    const congestion = await this.analyzeNetworkCongestion();

    // Get historical fee data
    const historicalFees = await this.getHistoricalFeeData(24); // Last 24 hours

    // Calculate base fee
    const baseFee = await this.calculateBaseFee(transaction);

    // Apply optimization strategy
    let optimizedFee: number;

    switch (strategy) {
      case 'CONSERVATIVE':
        optimizedFee = this.applyConservativeStrategy(baseFee, congestion, historicalFees);
        break;
      case 'BALANCED':
        optimizedFee = this.applyBalancedStrategy(baseFee, congestion, historicalFees);
        break;
      case 'AGGRESSIVE':
        optimizedFee = this.applyAggressiveStrategy(baseFee, congestion, historicalFees);
        break;
      case 'PREDICTIVE':
        optimizedFee = await this.applyPredictiveStrategy(baseFee, historicalFees);
        break;
      default:
        optimizedFee = baseFee;
    }

    return {
      baseFee,
      optimizedFee,
      strategy,
      congestion,
      estimatedConfirmationTime: this.estimateConfirmationTime(optimizedFee, congestion),
      successProbability: this.calculateSuccessProbability(optimizedFee, historicalFees),
    };
  }

  private async analyzeNetworkCongestion(): Promise<'low' | 'medium' | 'high'> {
    const samples = await this.connection.getRecentPerformanceSamples(5);
    const avgTPS = samples.reduce((sum, s) => sum + s.numTransactions, 0) / samples.length;

    if (avgTPS > 5000) return 'high';
    if (avgTPS > 2000) return 'medium';
    return 'low';
  }

  private applyConservativeStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const minHistoricalFee = Math.min(...historicalFees);
    return Math.max(baseFee, minHistoricalFee * 0.8);
  }

  private applyBalancedStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const medianFee = this.calculateMedian(historicalFees);
    const congestionMultiplier = congestion === 'high' ? 2.0 :
      congestion === 'medium' ? 1.5 : 1.0;
    return medianFee * congestionMultiplier;
  }

  private applyAggressiveStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const avgFee = this.calculateAverage(historicalFees, 95);
    return Math.max(baseFee * 1.5, avgFee);
  }

  private async applyPredictiveStrategy(
    baseFee: number,
    historicalFees: number[]
  ) {
    // Predictive logic based on historical trends and network state
    // ...
  }
}
```

# Key Takeaways

## Advanced Features Benefits

- **Smart Accounts:** Programmatic control with enterprise-grade security
- **MPC Wallets:** Distributed security for institutional use cases
- **SVM Execution:** High-performance parallel processing
- **DEX Integration:** Seamless token swap capabilities
- **Real-Time Events:** Live blockchain monitoring and notifications
- **Fee Optimization:** Cost-effective transaction processing

## Production-Ready Capabilities

- **Scalable Architecture:** Multi-instance deployment with load balancing
- **Security First:** Comprehensive validation and fraud prevention

- **Performance Optimized:** Caching, parallel processing, and efficient algorithms